Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

Kvízy v interaktívnych pracovných hárkoch pre logiku

Bakalárska práca

2025 Matej Gornál

Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

Kvízy v interaktívnych pracovných hárkoch pre logiku

Bakalárska práca

Študijný program:	Aplikovaná informatika
Študijný odbor:	Informatika
Školiace pracovisko:	Katedra aplikovanej informatiky
Školiteľ:	Mgr. Ján Kľuka, PhD.

Bratislava, 2025

Matej Gornál

80394254

Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta: Matej Gornál
Študijný program: aplikovaná informatika (Jednoodborové štúdium, bakalársky I. st., denná forma)
Študijný odbor: informatika
Typ záverečnej práce: bakalárska
Jazyk záverečnej práce: slovenský
Sekundárny jazyk: anglický

Názov: Kvízy v interaktívnych pracovných hárkoch pre logiku
Quizzes in interactive logic worksheets

Anotácia: Pri výučbe Logiky pre informatikov využívame interaktívne pracovné hárky, spravované webovou aplikáciou, ktorá vznikla v rámci bakalárskej práce M. Moka (2022) a ktorá integruje moduly na riešenie špecifických druhov cvičení. Niektoré teoretické otázky je však vhodné testovať formou kvízov. Tie zatiaľ zadávame ako voľný text, v ktorom študentstvo vyznačuje odpovede a vyučujúci/-e ich musia kontrolovať manuálne.

Cieľ: Vyvinúť webovú aplikáciu na:
• správu banky kvízových otázok viacerých typov,
• zostavovanie kvízov,
• vyhodnocovanie riešení
a modul pre webový pracovný hárok umožňujúci vypracovávanie kvízov zložených z vybraných otázok.

Literatúra: Mok, Matej: Interaktívny pracovný zošit pre výučbu logiky pre informatikov. Bakalárska práca. Bratislava : Univerzita Komenského, 2022.
Moodle community: Moodle Documentation. [online] https://docs.moodle.org/

Vedúci: Mgr. Ján Kľuka, PhD.
Katedra: FMFI.KAI - Katedra aplikovanej informatiky
Vedúci katedry: doc. RNDr. Tatiana Jajcayová, PhD.

Dátum zadania: 01.10.2024

Dátum schválenia: 29.10.2024

doc. RNDr. Damas Gruska, PhD.
garant študijného programu

.....
študent

.....
vedúci práce

Poďakovanie: Ďakujem môjmu školiteľovi Mgr. Jánovi Kľukovi, PhD. za pravidelné konzultácie a cenné rady, ktoré mi pomáhali pri zaznamenávaní pokroku počas celej práce.

Abstrakt

V tejto práci sa venujeme vývoju kvízového systému, ktorý bude použitý na vytváranie kvízov a následné testovanie študentov kvízovou formou v rámci predmetu Logika pre Informatikov. Implementujeme rozhranie v ktorom sa budú dať vypĺňať kvízy a následne modul, ktorý bude pridaný do aplikácie Logic Workbook. Naša aplikácia je prepojená s GitHubom prostredníctvom ktorého sa používatelia autentifikujú v rámci našej aplikácie a ukladajú si ich riešenia kvízov. V našom rozhraní sa kombinuje neformátovaný text s formátovaným matematickým textom, čím sa umožňuje písanie matematických formúl a dôkazov.

Na vývoj aplikácie sme používali knižnice React a Flask.

Aplikáciu budú testovať študenti v rámci predmetu, a neskôr bude nasadená do aplikácie Logic Workbook.

Kľúčové slová: GitHub, React, Flask, kvíz, Logic Workbook

Abstract

In this thesis, we present the development of a quiz system designed for creating and administering quizzes in the course Logic for Computer Scientists. The system consists of a user interface for solving quizzes and a module that integrates directly into an interactive Logic Workbook.

The application is integrated with GitHub for user authentication and for saving quiz solutions. The user interface supports a combination of plain text and formatted mathematical notation, enabling users to write mathematical formulas and proofs.

The system was developed using React for the frontend and Flask for the backend. It will be tested by students enrolled in the course and later deployed as a component of the interactive Logic Workbook.

Keywords: GitHub, React, Flask, quiz, Logic Workbook

Obsah

Úvod **1**

1 Východiská práce **3**

1.1 Úvod do problematiky 3

1.1.1 Testy vo vyučovaní 3

1.1.2 Typy testových úloh 3

1.1.3 Ciele použitia kvízov 4

1.2 Kvízové aplikácie 4

1.2.1 Moodle 4

1.2.2 Google Forms 5

1.2.3 Obmedzenia existujúcich riešení 6

2 Porovnanie frameworkov **7**

2.1 Framework Flask 8

2.2 Framework Django 10

2.3 Framework Express.js 11

2.4 Porovnanie frameworkov Flask, Django a Express 12

3 Použité technológie **15**

3.1 Flask 15

3.2 HTML, CSS, Bootstrap, React 15

3.3 Docker 17

3.4 PostgreSQL a pgAdmin 4 17

4 Návrh **19**

4.1 Funkčné požiadavky 19

4.1.1 Správa otázok 20

4.1.2 Správa kvízov 21

4.1.3 Správa študentov 25

4.1.4 Vývojové prostredie 25

4.2 Používateľské rozhranie 25

4.3 Používateľské roly 26
4.4 Návrh databázy 27
4.4.1 Dátový model 27

5 Implementácia **33**
5.1 Štruktúra 33
5.2 Databáza 35
5.3 Implementácia používateľského rozhrania 36
5.4 Nasadenie aplikácie 45
5.4.1 Autentifikácia cez GitHub pomocou OAuth 45
5.4.2 Nastavenie premenných prostredia 46
5.4.3 Build a compose projektu 46
5.5 Implementácia modulu pre Logic Workbook 47
5.5.1 Postup pridania modulu 47
5.5.2 Rozhranie pre Logic Workbook 48

6 Testovanie **51**

Záver **53**

Zoznam obrázkov

4.1 Náčrt základného rozhrania 26
4.2 Náčrt pre rozhranie otázok 27
4.3 Use case diagram 28
4.4 Triedny diagram pre Questions 28
4.5 Triedny diagram pre quizzes 31

5.1 Prihlásenie do systému 37
5.2 Rozhranie kvízy 38
5.3 Nastavenia šablóny kvízu 39
5.4 Pridávanie otázok do šablóny 40
5.5 Rozhranie pre evidenciu otázok 41
5.6 Rozhranie pre vytváranie otázok 42
5.7 Rozhranie pre vytvorenie kategórie 42
5.8 Správa používateľov 43
5.9 Štatistika študenta 43
5.10 Vygenerovaný kvíz 44
5.11 Kontrola odpovedí kvízu 45
5.12 Modul v interaktívnom pracovnom hárku 49

Zoznam tabuliek

2.1 Porovnanie Flask, Django a Express na základe knižnice alebo vstavanej funkcionality pre dané kritérium 13
2.2 Porovnanie Flask, Django a Express na základe vhodnosti pre vybrané kritériá 13

Úvod

Výučba predmetu logika pre informatikov prebieha z veľkej častí prostredníctvom nástroja Logic Workbook. Tento nástroj umožňuje zadávať cvičenia formou interaktívnych pracovných hárkov, ktoré môžu obsahovať textové a aplikačné komponenty. V aplikačných komponentoch môžu študenti riešiť rôzne typy úloh s automatickou kontrolou typov úloh, ako sú formalizácie, formálne dokazovanie a tvorba prvoradových štruktúr. Všetky tieto aplikácie vznikli v rámci bakalárskych prác Mateja Moka [19], Milana Cifru [20], Samanthy Gombarovej [21], Norberta Jurika [22] a Alexandry Nyitraiovej [23]. Pomocou týchto nástrojov sa študenti učia aplikovať preberané učivo na konkrétnych problémoch. Chceme však motivovať študentov k tomu, aby sa učili teóriu priebežne. Na to sme doteraz používali jednoduché otázky s viacerými možnosťami, ktoré boli v hárku reprezentované ako textové bunky. Tie sa vyhodnocovali ručne, alebo sa nechali študentom na samo-vyhodnotenie. Lepším riešením by bolo, aby sa hárok rozšíril o možnosť vkladania testových otázok alebo viac-otázkových testov, ktoré by sa vyhodnocovali automaticky alebo poloautomaticky.

Existuje viacero aplikácií, ktoré umožňujú testovanie študentov formou kvízových otázok. Medzi tieto aplikácie patria Moodle, Google forms . Tieto nástroje však majú samostatné rozhrania, na ktoré sa treba samostatne pripájať a prechádzať zo stránky na stránku. My chceme systém, ktorý sa bude dať integrovať priamo do existujúceho systému, do aplikácie Logic Workbooku. Študenti sa v takom prípade nebudú musieť prihlasovať do viacerých aplikácií a zároveň uvidia teoretické otázky a súvislosti priamo v hárku, v ktorom to práve riešia.

Preto sme sa rozhodli vyvinúť aplikáciu, ktorá umožní vytvárať a vypĺňať testy priamo v aplikácií Logic Workbook, bez potreby chodiť na iné stránky. Rozhranie bude rozdelené na dve časti, keď jedna bude ako samostatná doména pre učiteľa na vytváranie kvízov a v prípade potreby aj na vypĺňanie kvízov. Druhá časť bude samostatný komponent, ktorý sa pridá do aplikácie Logic Workbook, a umožní študentom pracovať s kvízmi priamo tam. Táto aplikácia bude ďalším rozšírením logického hárku, ktorý už teraz obsahuje viacero rozšírení.

V 1. kapitole predstavíme stručný úvod do problematiky a predstavíme podobné aplikácie, v ktorých sa dajú riešiť kvízové zadania. Ďalej (2. kapitola) v ktorej sa detailne pozrieme na výber frameworku, ktorý budeme používať na programovanie backendu

a frontendu. V 3. kapitole popíšeme technológie, ktoré sme si vybrali pre vývoj našej aplikácie. Potom v 4. kapitole prejdeme na návrh aplikácie, kde si rozoberieme funkčné požiadavky na systém, načrtneme si používateľské prostredie, predstavíme aký používatelia sú v našom systéme, ukážeme a vysvetlíme podrobný návrh databázy. V 5. kapitole sa pozrieme na implementáciu aplikácie, kde podrobne popíšeme jej štruktúru, akým spôsobom komunikuje databáza s aplikáciou, ako funguje autentifikácia cez Github a ako fungujú databázové migrácie. Tu predstavíme aj postup, akým sa dá aplikácia pridať ako modul do aplikácie Logic Workbook. Popíšeme si aj ako vyzerajú dané časti aplikácie a vysvetlíme čo sa kde dá robiť. V poslednej 6. kapitole si prejdeme spôsob testovania aplikácie a spätnú väzbu od študentov, ktorý ju testovali.

Kapitola 1

Východiská práce

1.1 Úvod do problematiky

1.1.1 Testy vo vyučovaní

Didaktický test je druh skúšky. Zväčša obsahuje uzatvorené otázky s pevnou štruktúrou. Menej často obsahuje otázky s otvorenými úlohami. Za test sa nepovažuje hocijaký papier s úlohami a otázkami. Didaktický test má byť obrazom učebného procesu. Použitie testu je veľký kompromis, lebo oproti ústnej skúške obsahuje obmedzené množstvo možností pokiaľ ide o skúšanie. Napriek tomu existuje viacero predností testov, v porovnaní s ostatnými. Hlavnými prednosťami sú, že pomocou testu dokážeme vyskúšať celú skupinu žiakov naraz, a pokiaľ sú testy v vypĺňané v softvéri, aj automaticky vyhodnotiť. [24]

Často sa využíva sumatívna skúška, kde overíme a zdokumentujeme, čo sa študent za dané obdobie naučil a podá o tom správu. Dôraz sa kladie na výstupný obraz(sumu) žiakových vedomostí. Pri tomto spôsobe sa na chyby pozerá ako na zlyhanie. [24]

Pomocou testu však vieme aj analyzovať a skúmať postupy žiakov a ako dospeli ku správnym a nesprávnym výsledkom. V prípade nesprávnych výsledkov vieme študenta ďalej nasmerovať k tomu, aby v budúcnosti dospel ku správnym odpovediam. Tento spôsob testovania nazývame formatívny. Tento spôsob je viac uvoľnený a zameriava sa prednostne na to, aby žiak vedel ako na tom je. Výsledok sa neklasifikuje a je len na žiakovi a učiteľovi ako so získanými infomáciami z testu naloží.[24]

Rozdiely medzi sumatívnym a formatívnym hodnotením nie sú v tom , ako sa získavajú informácie o úspešnosti študentov, ale v tom, ako sa tieto dáta spracujú. [24]

V našej práci budeme didaktické testy nazývať kvízy.

1.1.2 Typy testových úloh

Rozdeľujeme 2 typy testových úloh a to otvorené a uzavreté. [24]

Otvorené úlohy sú také, kde študent priamo píše svoju odpoveď, teda nemá možnosť výberu z pred-pripravených odpovedí, ale musí ju vytvoriť sám. [24]

Uzavreté úlohy sú také, kde si študent vyberá z možností odpovedí, ktoré má v ponuke. [24] Každú uzavretú úlohu vieme formulovať viacerými spôsobmi. Uzavreté úlohy rozdeľujeme na tieto typy:

- **Binárne úlohy**, pri ktorých vyberám z dvoch možností(áno/nie, pravda/nepravda,...) [24]
- **Úlohy s výberom viacerých možností**, pri ktorých je buď jedna alebo viacero správnych odpovedí. [24]
- **Priraďovacie úlohy** v ktorých má žiak dva stĺpce a spája možnosti, ktoré ku sebe patria. [24]

V našich kvízoch budeme používať aj otvorené otázky a aj uzavreté otázky, ktoré sme spomínali.

1.1.3 Ciele použitia kvízov

Na predmete Logika pre informatikov je naším hlavným cieľom používať kvízy formatívnym spôsobom. Využívaním kvízov chceme študentov motivovať k používaniu správnej terminológie a venovaniu pozornosti porozumeniu definíciám a dôležitým vetám. Pomocou kvízu si študent overí, či im naozaj pochopil a vedel by ich použiť aj v okrajových alebo zložitejších prípadoch ako sú tie, ktoré sa bežne vyskytujú v úlohách, ktoré sa riešia na cvičeniach.

1.2 Kvízové aplikácie

V tejto sekcií porovnáme Moodle a Google Form, ktoré poskytujú možnosť vytvárania kvízov. Popíšeme aj existujúce obmedzenia aplikácií, ktoré by sme v našej chceli vyriešiť a tým sa odlíšiť od existujúcich riešení.

1.2.1 Moodle

Moodle je systém na podporu výučby(learning management system), ktorý umožňuje tvorbu a správu online kurzov [1]. Tieto kurzy môžu ako jednu z aktivít obsahovať kvízy. Tie sú zostavené z otázok, ktoré sa v Moodli dajú vytvárať a organizovať v banke otázok a neskôr využívať v kvízoch.

[1] `https://moodle.org`

Banka otázok Všetky otázky sú ukladané v banke otázok, a každá otázka môže byť priradená do kategórie. Otázky sa dajú editovať a pridávať im nastavenia podľa daného typu otázky.

Výhody

- **Flexibilita** systém umožňuje vytvárať rôzne typy kurzov, kvízov, podporuje rôzne formy učenia
- **Nástroje na hodnotenie** systém poskytuje nástroje na sledovanie pokroku študentov, tak isto ako aj možnosť spätnej väzby od študentov.
- **Podpora formátov** systém podporuje videá, zvuky a rôzne interaktívne prvky

Nevýhody

- **Nepodporuje latex renderovanie** systém nepodporuje renderovanie latex výrazov, čo je pre náš systém veľmi dôležité, keďže pri kvízoch v predmete Logika pre informatikou je hlavnou súčasťou kvízov text, ktorý je napísaný pomocou markdownu a latexu.
- **Vlastné makrá** pre zjednodušenie písania markdownu a latexu môžu vznikať makrá, ktoré umožnia pomocou predpísaných pravidiel písať zložitejšie formuly jednoduchšie. Podpora na toto v Moodle nie je.
- **Integrácia aplikácie** kvízový systém treba integrovať priamo do aplikácie Logic Workbook a Moodle nepodporuje toto prepojenie, takže sa nedá preniesť do našej aplikácie ako samostatný modul.

1.2.2 Google Forms

Google Forms je bezplatný online nástroj od spoločnosti Google na tvorbu formulárov a prieskumov, čo sa dá simulovať ako kvízy. Umožňuje používateľom jednoducho vytvárať formuláre a vypĺňať ich.

Výhody

- **Flexibilita** systém umožňuje širokú prispôsobiteľnosť typov otázok a odpovedí, ako aj prispôsobenie vzhľadu formulára.
- **Spätná väzba** systém umožňuje po odoslaní formulára študentom okamžite vidieť svoje odpovede a výsledky, čo umožňuje rýchlu spätnú väzbu a interakciu

Nevýhody

- **Nepodporuje latex renderovanie** systém nepodporuje renderovanie latex výrazov, čo je pre náš systém veľmi dôležité, keďže pri kvízoch v predmete Logika pre informatikou je hlavnou súčasťou kvízov text, ktorý je napísaný pomocou markdownu a latexu.

- **Integrácia aplikácie** kvízový systém treba integrovať priamo do aplikácie Logic Workbook a Google Forms nepodporuje toto prepojenie, takže sa nedá preniesť do našej aplikácie ako samostatný modul.

1.2.3 Obmedzenia existujúcich riešení

Pri analýze systémov Moodle a Google Forms sme zistili, že tieto riešenia obsahujú obmedzenia, ktoré sú pri výučbe predmetu logika pre informatikov potrebné.

- **Integrácia do aplikácie Logic Workbook**, čo je jedným z hlavných cieľov tejto práce nie je podporovaná ani v jednom spomínanom systéme. My potrebujeme vytvoriť modul, ktorý sa bude dať pridať do Workbooku a študenti budú môcť pracovať s kvízmi priamo tam.

- **Spätná väzba**, okrem klasickej spätnej väzby, v našej aplikácií je potrebná špecifickejšia možnosť spätnej väzby ako zo strany učiteľov tak aj študentov.

- **Banka otázok** musí byť navrhnutá tak, aby podporovala začleňovanie otázok do kategórií, ktoré tvoria stromovú štruktúru. Otázky majú svoj typ, a pri každej zmene údajov o otázke sa vytvára nová verzia otázky. V prípade potreby sa dá ku staršej verzií vrátiť.

- **Absencia podpory pre vlastné makrá**, pre zjednodušenie písania v jazyku Markdown, potrebujeme využícať preddefinované makrá, ktoré zrýchľujú písanie.

Kapitola 2

Porovnanie frameworkov

Webové aplikácie sa zvyčajne skladajú zo serverovej časti (backendu) a častí vykonávanej webovým prehliadačom (frontendu). Tieto časti medzi sebou zvyčajne komunikujú pomocou protokolu HTTP. Backend poskytuje frontendu na získavanie dát a vykonávanie zmien poskytuje aplikačné rozhranie (API). Častým typom API rozhrania je REST, ktoré je špecifické tým, že dátové objekty a operácie sú reprezentované URL adresami a HTTP požiadavkami na server, ktoré sú rôzneho typu (GET, POST, DELETE). Dáta sa pri požiadavkách na server posielajú nie vo formáte HTML stránok (pre ktoré bol pôvodne protokol HTTP navrhnutý), ale v jednoduchom štruktúrovanom formáte JSON.

Backend môže byť vytvorený v rôznych jazykoch, ale pre frontend je typické, že je vytvorený v JavaScripte. V súčastnosti existujú aj iné jazyky, ale tie sú neskôr kompilované do JavaSriptu. Pre jednoduchší vývoj webových aplikácií vzniklo viacero frameworkov, čo sú knižnice ktoré vytvoria kostru aplikácie a prepisujú jej štruktúru. Programátor môže pridať nové funkcionality do nej.

Backendové frameworky poskytujú rozhranie na zjednodušenie prístupu ku databáze, ktoré poskytuje na dáta pohľad, ako by boli objekty a volá sa ORM. Niektoré frameworky poskytujú nad vytvoreným ORM generovať REST rozhranie, ktoré poskytuje základné CRUD operácie.

CRUD operácie sa používajú pri práci s databázami a predstavujú Create, Read, Update, Delete funkcionalitu nad dátami.

Niektoré frameworky majú vstavané jednoduché administratívne rozhranie, ktoré je realizované predovšetkým pomocou CRUD operácií.

Niektoré frameworky poskytujú aj správu používateľov a overovanie ich totožnosti pomocou externej autentifikácie a JWT tokénu. JWT (JSON Web Token) obsahuje informácie o používateľovi a je digitálne podpísaný.

Backend webovej aplikácie treba rozumne členiť na menšie časti, kvôli prehľadnosti kódu a organizácií do menších častí, čo nám zabezpečujú mikroservisy. Mikroservisy

komunikujú medzi sebou cez API a každý ma zodpovednosť za nejakú menšiu časť funkcionality.

Vo väčších webových aplikácia sa často stáva, že sa opakovane tvoria niektoré časti stránky. Na zjednodušenie a oddelenie dizajnu od aplikácie slúžia šablónovacie knižnice, pomocou ktorých vieme dynamicky vkladať údaje do HTML štruktúr.

Na základe spomenutých vlastnosti a funkcionalít, ktoré by mal ako backend poskytovať sa pozrieme na porovnanie troch frameworkov, ktoré sa môžu použiť na jeho vybudovanie. Sú nimi Flask, Django, ktoré sú frameworky v programovacom jazyku Python a Express.js, ktorý je framework v Javascripte. Tieto tri frameworky budeme porovnávať na základe špecifických kritérií. Dôraz budeme klásť na to, či daný framework má vlastné vstavané ORM, a pokiaľ nemá tak pomocou akej zvyčajnej knižnice túto službu poskytuje. Ďalej na základe toho, akú má podporu pre REST API a automatizáciu CRUD operácií. Zaujímať nás bude taktiež či daný framework obsahuje administratívne rozhranie a či ho vie automaticky generovať. Porovnávať budeme aj na základe možnosti implementácie mikroservisov, integrácie externej autentifikácie (JWT) a podpory pre templating engines. V poslednej podkapitole vytvoríme tabuľku ktorá všetky tieto informácie zhrnie do jedného celku na základe ktorých si vyberieme vhodný framework na vývoj aplikácie.

2.1 Framework Flask

Flask je mikro webový framework, napísaný v jazyku Python [1]. Vznikol ako prvoaprílový žart, ktorý sa postupom času stal veľmi používaný a obľúbený spomedzi Python webových frameworkov [1]. Je jedným z najviac používaných startupmi a je ideálny pre jednoduché riešenia. Medzi aplikácie Flasku patria napríklad Pinterest a Lindeln [1]. Tento framework je zadarmo a má aktívnu komunitu programátorov, kvalitnú dokumentáciu. Vďaka jednoduchému a flexibilnému použitiu je veľmi obľúbený. [1].

Podpora ORM

Flask nemá vstavanú podporu pre ORM (Objektovo relačné mapovanie), avšak existujú knižnice, ktoré dokážu zabezpečiť ORM prístup [4]. Najpopulárnejšou knižnicou na to je SQLAlchemy vytvorená pre Flask. Táto knižnica ponúka intuitívny interface pre databázové operácie vo Flask frameworku. V SQLAlchemy sú databázové štruktúry reprezentované ako Python triedy. Toto je príklad Python kódu, ktorý využíva knižnicu SQLAlchemy a definuje triedu User, ktorá je v našej databáze tabuľkou:

```
class User(db.Model):
    id = db.Column(db.Integer, primary_key = True)
```

```
    username = db.Column(db.String(80), unique=True, nullable=False)
    email = db.Column(db.String(120), unique=True, nullable=False)

    def __repr__(self):
        return '<User␣%r>' % self.username
```

REST API - automatizácia CRUD

Vo Flask-u dokážeme vytvoriť REST API pre automatizáciu CRUD operácií pomocou kombinácie viacerých knižníc [4]. Môžeme použiť SQLAlchemy knižnicu na prácu s databázou a Marshmallow knižnicu na serializáciu a validáciu dát [4]. Toto nám umožní vytvoriť efektívnu implementáciu API, ktorá bude komunikovať s databázou pomocou CRUD operácií. Kombinácia týchto knižníc taktiež umožňuje prevod dát z tabuliek do formátu JSON, čím poskytuje vysokú flexibilitu.

Administratívne rozhranie

Administratívne rozhranie je možné vytvoriť pomocou knižnice Flask-Admin, ktorá dokáže vytvoriť toto rozhranie na základe existujúceho dátového modelu [5]. Poskytuje používateľsky prívetivé rozhranie na manažovanie dát webovej servisy.

Implementácia mikroservisov

Flask podporuje mikroservisy a je veľmi vhodný pokiaľ sú potrebné [1].Keďže Flask je mikro webový framework, implementácia mikroservisov je veľmi flexibilná a môžeme si vybrať aké komponenty použiť a neexistuje presne daná štruktúra projektu, čo napomáha vytvárať mikroservisy.

Podpora externej autentifikácie JWT

Pre externú autentifikáciu existuje knižnica Flask-JWT-Extended, ktorá umožňuje jednoduché a efektívne spravovanie autentifikácie a autorizácie pomocou JSON Web Tokens. [2]

Šablónovacia knižnica

Najpopulárnejší templating engine, ktorý sa využíva v súvislosti s Flask frameworkom je Jinja2 [1]. Keďže je Flask mikro webový framework, využívanie templating engines je jedna z najdôležitejších vlastností. Jinja2 sa využíva na dynamické generovanie obsahu.

2.2 Framework Django

Django je vysoko úrovňový webový framework využívajúci programovací jazyk Python. Bol vyvinutý skúsenými vývojármi, a tento framework umožňuje rýchly vývoj a bezpečnosť stránky [6]. Framework je zadarmo a je open-source, teda má aktívnu komunitu a výbornú dokumentáciu [6].

Podpora ORM

Django ORM je vstavaná funkcionalita tohoto frameworku na používanie ORM prístupu [6]. Je to jeden z najdôležitejších prvkov Djanga, ktorý umožňuje komunikáciu s databázou pomocou objektovo orientovaného prístupu. Django na to využíva vstavanú funkcionalitu Django models. Tu je ukážka kódu, ako sa dá v Django models vytvoriť trieda:

```
from django.db import models

class Model(models.Model):
    title = models.CharField(max_length = 200)
    description = models.TextField()
```

REST API - automatizácia CRUD

Vytvorenie REST API s automatizáciou CRUD je vo frameworku Django jednoducho realizovateľné pomocou vstavanej knižnice Django REST framework (DRF) [6]. Daný nástroj poskytuje všetko potrebné na vytvorenie funkčného REST API s podporou CRUD operácií. DRF obsahuje komponenty ako Serializer, ViewSet, vďaka ktorým je implementácia veľmi rýchla.

Administratívne rozhranie

Django poskytuje robustné vstavané administratívne rozhranie, ktoré umožňuje správu dátového modelu [6]. Toto rozhranie je jedným z najtypickejších črtov tohto frameworku. Rozhranie sa automaticky generuje na základe definovaných modelov. Nájdeme ho na http://127.0.0.1:8000/admin/ a prihlásime sa pomocou superpoužívateľa. Taktiež ponúka aj možnosť viacerých rozšírení rozhrania na základe potreby.

Implementácia mikroservisov

Keďže je Django high-level webový framework, implementácia mikroservisov môže byť komplikovanejšia a náročnejšia [6]. Django síce ponúka veľké portfólio funkcionality pre rýchly vývoj, ale pre menšie aplikácie môže byť pri implementácií mikroservisov

ťažkopádne. Môže používať Django REST Framework (DFR) ktorá slúži na API služby ale môže byť základom pre mikroservisy.

Podpora externej autentifikácie JWT

Externú autentifikáciu zabezpečuje pomocou Django REST Framework kde využíva balík djangorestframework-simplejwt.

Šablónovacia knižnica

Django používa Django Template Language (DTL), ktorý je vstavaný vo frameworku [6]. Django ponúka silný templating engine, vďaka ktorému môžeme efektívne generovať HTML obsah.

2.3 Framework Express.js

Express.js je mikro webový framework, ktorý slúži na budovanie webových stránok [8]. Je to multiplatformový Javascript runtime, ktorý umožňuje spúšťanie kódu napísaného v Javascript-e na serveri za pomoci Node.js [8]. Tento framework používa asynchrónnosť, aj preto sa využíva napríklad na chatové aplikácie. Je veľmi obľúbený pre jeho flexibilitu. Populárnym sa stal hlavne kvôli tomu, že Javascript je veľmi populárny, a preto sa na tento framework vedelo adaptovať veľa programátorov veľmi jednoducho [8].

Podpora ORM

V Express.js frameworku nie je priamo vstavaná ORM funkcionalita, ale existuje viacero populárnych knižníc, ktoré dokážu garantovať tento prístup. Jedna z najpopulárnejších knižníc pre tento framework na dosiahnutie ORM prístupu je Sequelize knižnica. Sequelize je moderná knižnica pre ORM prístup ku databázam Oracle, Postgres, MySQL. Tu je ukážka ako sa dá knižnica importovať a pomocou nej vytvoriť triedu:

```
import { Sequelize, DataTypes } from 'sequelize';

const sequelize = new Sequelize('sqlite::memory:');
const User = sequelize.define('User', {
  username: DataTypes.STRING,
  birthday: DataTypes.DATE,
});
```

REST API - automatizácia CRUD

Express.js podporuje REST API s automatizáciou CRUD operácií, za pomoci knižnice Sequelize, ktorý slúži ako ORM. Za využitia Express.js na spracovanie HTTP požiadaviek a Sequelize na prácu s databázami ako Postgres, Oracle, MySQL dokážeme vytvoriť funkčnú API.

Administratívne rozhranie

Na vytvorenie rozhrania sa používa knižnica AdminJS, ktorá poskytuje prispôsobiteľné rozhranie pre aplikácie postavené na Node.js a Express.js. Dokážeme ho aj sami rozširovať a pridávať funkcionalitu, ktorú potrebujeme.

Implementácia Mikroservisov

Implementácia mikroservisov pomocou Express.js je veľmi obľúbená pre mnohé webové aplikácie. Ako mikro webový framework sa veľmi hodí na vytváranie mikroservisov. Implementácia je priama, bez potreby ďalších rozšírení.

Podpora externej autentifikácie JWT

Express.js využíva na externú autentifikáciu knižnicu jsonwebtoken. Pomocou tejto knižnice sa dajú jednoducho generovať a overovať tokény.

Šablónovacia knižnica

Express.js používa na templating engines knižnicu EJS, ktoré poskytujú nástroje na dynamické generovanie HTML obsahu.

2.4 Porovnanie frameworkov Flask, Django a Express

Následujúce tabuľky sumarizujú vhodnosť použitia daného frameworku z pohľadu aplikácie, ktorú chceme vyvinúť. Tabuľka 2.1 popisuje knižnice, ktoré daný framework používa na riešenie danej funkcionality. Pokiaľ má framework danú funkcionalitu už vbudovanú, v tabuľke je to popísané ako vlastné, inak je napísaný názov knižnice, ktorá poskytuje danú funkcionalitu. Tabuľka 2.2 poskytuje pohľad na vhodnosť použitia danej funkcionality z pohľadu aplikácie, ktorú ideme vyvinúť. V tabuľke sa objavujú hodnoty vhodné, nevhodné a stredne vhodné.

Používanie Express.js považujeme za problematické, lebo na implementáciu ORM používa viaceré knižnice a neexistuje jednoznačné odporúčanie, ktoré ORM by sa malo používať. Express.js taktiež vyžaduje manuálnu správu pripojení ku databáze, čo môže

Tabuľka 2.1: Porovnanie Flask, Django a Express na základe knižnice alebo vstavanej funkcionality pre dané kritérium

Funkcia	**Flask**	**Django**	**Express**
ORM	SQLAlchemy	Vlastné	Sequelize
REST API	SQLAlchemy, Marshmallow	Vlastné	Sequelize
Administratívne rozhranie	Flask-Admin	Vlastné	AdminJS
Mikroservisy	Vlastné	Vlastné	Vlastné
Externá autentifikácia JWT	Flask-JWT-Extended	Vlastné	jsonwebtoken
Šablonovacia knižnica	Jinja2	Vlastné	EJS

Tabuľka 2.2: Porovnanie Flask, Django a Express na základe vhodnosti pre vybrané kritériá

Funkcia	**Flask**	**Django**	**Express**
ORM	Vhodné	Vhodné	Nevhodné
REST API	Vhodné	Vhodné	Stredne
Administratívne rozhranie	Vhodné	Vhodné	Stredne
Mikroservisy	Vhodné	Stredne	Vhodné
Externá autentifikácia JWT	Vhodné	Vhodné	Vhodné
Šablónovacia knižnica	Vhodné	Vhodné	Stredne

komplikovať kód a v prípade zlého spracovania zavretia spojenia procesy, ktoré budu na databáze bežať dlhú dobu.

Django považujeme za stredne vhodné na použitie mikroservisov kvôli tomu, že je ťažkopádnejší v porovnaní s ostatnými frameworkami kvôli tomu, že je vyvinuté na správu veľkých a mohutných aplikácií.

Kapitola 3

Použité technológie

V tejto kapitole si zhrnieme všetky technológie ktoré sme vybrali na vývoj aplikácie a v krátkosti ich popíšeme.

3.1 Flask

Na základe porovnania technológií v sekcií [2] sme sa rozhodli, že na vývoj backendu našej aplikácie budeme používať framework Flask, ktorý je ideálny na rýchly a priamočiary začiatok, pričom poskytuje možnosť fungovať aj pre komplexné aplikácie.

3.2 HTML, CSS, Bootstrap, React

HTML

HTML (HyperText Markup Language) je najzákladnejší jazyk určený na tvorbu webu [9]. Definuje význam a štruktúru webového obsahu. HTML dokumenty sú tvorené značkami, ktoré označujú jednotlivé prvky stránky a určujú ich vzhľad a štruktúru. HTML je základom pre tvorbu dynamických webových aplikácií, keďže je spravidla kombinovaný s ďalšími technológiami ktoré poskytujú štýl a interaktivitu.

HTML kód je štruktúrovaný tak, že na začiatku stránky musí byť deklarovaný !DOCTYPE a následne <html> tag, do ktorého sa dáva hlavička stránky (<head>) , telo stránky (<body>), do ktorého sa dáva celková štruktúra stránky, ktorá sa zobrazí používateľovi, a na koniec stránky sa dáva päta (<footer>).

CSS

(CSS) je jazyk na popis vzhľadu dokumentu napísaného v HTML [10]. CSS popisuje, ako by mali byť prvky zobrazené na obrazovke, na papieri, pri reči alebo na iných médiách.

CSS patrí medzi základné jazyky otvoreného webu a je štandardizovaný naprieč webovými prehliadačmi podľa špecifikácií W3C. Predtým sa vývoj rôznych častí špecifikácie CSS robil synchronizovane, čo umožnilo verziovanie najnovších odporúčaní. Existuje viacero verzií CSS ako sú CSS1, CSS2.1 alebo CSS3. Už ale nevznikne ďalšia verzia ako napríklad CSS4; namiesto toho je teraz všetko pod jednotným názvom "CSS". CSS sa dokáže zameriavať priamo na konkrétne pomocou triedy alebo id HTML prvku a priraďovať im štýl. [10]

Bootstrap

Bootstrap je najpopulárnejší open-source framework pre vývoj responzívnych a mobilných webových aplikácií [11]. Bol vytvorený spoločnosťou Twitter a je založený na HTML, CSS a JavaScriptu [11]. Bootstrap poskytuje predpripravené komponenty a šablóny, ako sú navigačné panely, formuláre, tlačidlá, tabuľky a ďalšie, ktoré uľahčujú tvorbu vizuálne konzistentných a funkčných stránok. Framework umožňuje rýchly vývoj webových aplikácií taktiež zabezpečuje automatické prispôsobenie rozloženia stránok rôznym veľkostiam obrazoviek.

Bootstrap tiež obsahuje množstvo prispôsobiteľných štýlov a interaktívnych prvkov, ako sú modálne okná, karusely a upozornenia, čo zjednodušuje implementáciu pokročilých funkcií. S Bootstrapom je vývoj responzívnych webových aplikácií rýchlejší a efektívnejší.

React

React je JavaScript framework využívaná na tvorbu používateľských rozhraní, vyvinutá spoločnosťou Meta [12]. React umožňuje vytvárať interaktívne a dynamické webové aplikácie pomocou komponentov, kde každý komponent predstavuje samostatnú časť používateľského rozhrania.

Dôležitou vlastnosťou Reactu je použitie virtuálneho DOM-u [12]. Virtuálny DOM (Document object model) je štrutúrovaná reprezentácia HTML dokumentu formou stromu, s ktorým vieme cez JavaScript manipulovať. Tento prístup umožňuje efektívne aktualizácie a renderovanie používateľského rozhrania, a preto vie React rýchlo nájsť zmeny a manipulovať čo najmenej s reálnym DOM-om, čo prispieva k väčšiemu výkonu aplikácie.

React je flexibilný a rozšíriteľný, čo uľahčuje jeho integráciu do rôznych projektov. Je veľmi populárny a často využívaný na tvorbu moderných webových aplikácií.

3.3 Docker

Docker je platforma, ktorá umožňuje spúšťanie aplikácie cez kontajnery [13]. Kontajnery sú podsystémy, ktoré sú izolované od ostatných častí operačného systému a obsahujú všetko čo je potrebné na beh danej aplikácie takže sa vie spustiť všade rovnako. To zabezpečuje možnosť spúšťať aplikáciu na rôznych prostrediach. Vďaka použitiu kontajnerov sa znižuje šanca na konflikty medzi aplikáciami, ktoré sú spustené na danom prostredí.

Docker obraz

Docker obrazy sú šablóny, podľa ktorých sa vytvárajú kontajneri [13]. Obsahujú kód, zoznam knižníc, nástroje potrebné na to, aby sa dala aplikácia spustiť. Tieto obrazy sú prenosné, čo im umožňuje, aby boli spúšťané na rôznych prostrediach. Pokiaľ je už obraz vytvorený, používateľ si vie stiahnuť aj hotový obraz.

DockerFile

Docker môže automaticky vytvárať obrazy čítaním inštrukcií z Dockerfile. Dockerfile je textový dokument, ktorý obsahuje všetky príkazy, ktoré by používateľ mohol zadať v príkazovom riadku na zostavenie obrazu.

Docker Compose

Docker Compose je nástroj na definovanie a spúšťanie aplikácií pozostávajúcich z viacerých kontajnerov [14]. Pri spustení compose sa vytvoria všetky docker obrazy podľa DockerFile-ov, ktoré sú v danom projekte definované.

3.4 PostgreSQL a pgAdmin 4

PostgreSQL je objektovo-relačný systém na správu databáz, známy svojou rozšíriteľnosťou a dodržiavaním štandardov [16]. Medzi jeho funkcie patrí podpora pre zložité dotazy, transakcie, integračné obmedzenia a viacúrovňové indexovanie.

pgAdmin 4 je najpopulárnejší open-source nástroj na správu a vývoj pre PostgreSQL [15]. Poskytuje grafické rozhranie pre správu databáz, čo uľahčuje tvorbu, údržbu a používanie databázových objektov. pgAdmin 4 je navrhnutý tak, aby vyhovoval potrebám nováčikov aj skúsených používateľov PostgreSQL.

Kapitola 4

Návrh

V návrhu sa budeme venovať tomu, ako vytvoriť požadovaný systém na základe týchto požiadaviek vyvinieme:

- Rozhranie na tvorbu a úpravu otázok
- Prostredie na vypĺňanie kvízov
- Prostredie na zostavovanie kvízov
- Prostredie, na sledovanie úspešnosti kvízov a otázok
- Prostredie na manuálne hodnotenie kvízov
- Modul na vypĺňanie kvízov priamo v aplikácií Logic Workbook

Dôležitou súčasťou systému je implementovať podporu pre zobrazovanie textu vo formáte Markdown a LaTeX, aby sme vedeli zobrazovať matematické texty. K tomu sa viaže aj potreba definovania vlastných LaTeX-ových makier, ktoré umožnia jednoduchšie písanie LaTeXkódu pomocou preddefinovaných príkazov.

V podkapitolách ďalej špecifikujeme funkčné požiadavky (4.1), navrhneme používateľské rozhranie (4.2), zadefinujeme používateľské roly (4.3) a navrhneme databázový model (4.4).

4.1 Funkčné požiadavky

Požiadavky definujú, ako systém funguje z hľadiska jeho používateľov. Zoznam v tejto sekcií poskytne požiadavky, ktoré je potrebné splniť z hľadiska funkcionality kvízového systému. Funkčné požiadavky na systém neboli od začiatku pevne dané, ale postupne sa iteratívnym procesom vyvíjali a pribúdali ako postupne systém vznikal.

4.1.1 Správa otázok

V tejto častí sa venujeme tomu, ako sa bude pracovať s otázkami v systéme a čo sa s nimi bude dať robiť.

Správa banky otázok

V banke otázok sa nám zobrazia všetky otázky. Pre predpoklad veľkého množstvá otázok, sú stránkované po 10 na jednu stranu, čo sa dá upraviť. Každá otázka obsahuje svoj názov, krátku časť textu a metadáta o tom, kedy bola vytvorené a kto ju vytvoril. Každý názov otázky sa dá stlačiť a tým editujeme otázku. Pri každej otázke je jej typ a možnosť kopírovať a archivovať otázku. Všetky otázky sú utriedené do kategórií, pomocou ktorých ich môžeme filtrovať. Po kliknutí na kategóriu zo stromu vyfiltrujeme otázky len danej kategórie a jej podkategórií. Ďalej vieme otázky usporiadať na základe toho kedy boli vytvorené a podľa abecedného poradia zostupne aj vzostupne. Otázky vieme aj filtrovať na základe toho kto ich vytvoril, a či sú aktívne, neaktívne alebo všetko dokopy. Aktívnosť otázky sa určuje na základe toho, či je archivovaná alebo nie.

Tvorba a úprava otázok

Otázky sa tvoria v samostatnom rozhraní. Tu si bude používateľ vyberať, do akej kategórie danú otázku zaradí, čo je dôležité pri tvorbe kvízov v prípade vyberania náhodných otázok a zaraďovania do stromovej štruktúry. Ďalej je potrebné vyplniť textové polia ako názov otázky, kde je umožnené písať len klasický text. Vypĺňanie polí textu otázky, negatívnej a pozitívnej spätnej väzby umožňuje písať aj matematický text pomocou Markdownu a LaTeX-u.

Ďalej treba vybrať typ otázky. Na výber sú 3 typy, krátka odpoveď, spájačka a výber viacerých možností, podľa čoho sa dynamicky zmení rozhranie na vyplnenie údajov o danej otázke.

Krátka odpoveď Potrebné je napísať správnu odpoveď na danú otázku. Očakáva sa, že ňou bude jedno alebo niekoľko slov. Učiteľ môže zadať odpoveď aj vo forme regulárneho výrazu, ktorý sa pri vyhodnotení porovná so študentovou odpoveďou. To umožňuje prijať napríklad synonymá správnej odpovede.

Spájačka Pri tomto type otázky tvoríme páry - dvojice, ktoré tvoria ľavú a pravú stránku spájačky. Nová dvojica sa môže vytvoriť až vtedy, keď aktuálna dvojica má aj pravú aj ľavú stranu. Pokiaľ je viac pravých strán s rovnakým názvom, spoja sa do jednej možnosti. Ku každému páru je možnosť pridať voliteľnú spätnú väzbu .Pri tomto type otázky ešte existuje možnosť použitia tzv. distraktorov. Distraktory slúžia na pridanie ľubovoľného počtu nesprávnych možností. Ich úlo-

hou je zvýšiť náročnosť otázky a zistiť, či študenti rozumejú danej otázke a nedajú sa rozptýliť.

Výber viacerých možností Pri tomto type otázky sa vypĺňa názov možností, a označuje sa, či je správna. Nová možnosť pribudne vtedy, keď sa vyplní aktuálna. Ku všetkým typom otázok sa dá ku každej možností pridávať voliteľnú spätnú väzbu, ktorý sa neskôr bude môcť zobrazovať v kvíze pri vyhodnotení.

Vytváranie a úprava otázok Po vytvorení otázky vznikne otázka a jej prvotná verzia automaticky. Každú otázku vieme upravovať, keď to je potrebné. Pri úprave otázky sa dajú meniť všetky hore popísané možností. Pri úprave otázky a jej uložení sa vytvorí nová verzia otázky, ktorá sa stane aktuálnou a bude sa zobrazovať v rozhraní.

Kopírovanie otázok Existuje aj možnosť kopírovať otázky. Funguje to rovnako ako upravovanie otázky len s rozdielom, že po uložení sa vytvorí úplne nová otázka s prvotnou verziou.

Sériové vytváranie otázok Táto možnosť funguje len keď vytvárame novú otázku. V rozhraní na vytváranie otázok bude zaškrtávacie políčko s textom "Create More", ktoré keď je aktívne, tak po uložení otázky nebudeme presmerovaný na rozhranie so zoznamom otázok, ale ostanú vyplnené údaje o otázke ktoré sme predtým zadali a po každom stlačení na uloženie otázky sa otázka uloží do banky otázok a používateľovi bude umožnené ďalej vytvárať otázky.

Správa kategórií

Kategórie organizujeme do štruktúry n-árneho stromu. To znamená že každá kategória je buď superkategória, alebo je podkategóriou nejakej inej kategórie. Kategória môže byť podkategóriou len jednej nadkategórie. Týmto nám vzniká stromová štruktúra, do ktorej môžeme ukladať otázky a pracovať s nimi. Aj v interaktívnom rozhraní sa tieto kategórie budú zobrazovať ako strom kategórií. Kategóriu vieme vytvoriť v osobitnom rozhraní, kde jej stačí priradiť nadkategóriu a jej názov. Pri tvorbe kategórie sa jej generuje aj slug, čo je očistení reťazec bez medzier a špeciálnych znakov. Ten sa generuje automaticky na základe napísaného názvu. Pokiaľ používateľ chce, môže tento slug upraviť aj manuálne, ale v takom prípade sa prestane generovať automaticky a bude ho musieť manuálne dopísať.

4.1.2 Správa kvízov

Správu kvízov budeme organizovať následovne, zadefinujeme šablóny a generovanie kvízov. Šablóna kvízu je predloha, podľa ktorej sa daný kvíz vygeneruje.

Zobrazenie kvízov

Pri zobrazení kvízov zobrazujeme názov kvízu, a rôzne metadáta o danom kvíze. Dávame informáciu, či sú sekcie kvízu zamiešané a koľko otázok a sekcií kvíz obsahuje. Ďalej zobrazujeme, od kedy do kedy je kvíz aktívny a od kedy bude možné zobrazovať spätnú väzbu ku danému kvízu. Podľa týchto kritérií sa zobrazujú tlačidlá, ktoré umožňujú zapnúť pokus kvízu, pozrieť si hodnotenie posledného kvízu a po uzatvorení kvízu pozrieť si konečné hodnotenie. Pokiaľ študent využil možnosť viacerých pokusov na vypracovanie kvízu, vie si pozrieť aj hodnotenia starších pokusov. Tieto funkcionality sú spoločné aj pre študenta aj pre učiteľa.

Učiteľ navyše po kliknutí na kôš dokáže archivovať daný kvíz. Po kliknutí na názov kvízu dokáže upravovať daný kvíz alebo pri kliknutí na tlačidlo vytvárať nový kvíz.

Tvorba novej šablóny kvízu

Pri tvorbe nového kvízu je nutnou podmienkou zadať názov kvízu, bez čoho kvíz nevie byť vytvorený. Učiteľ zadáva, či opakované pokusy na kvíz sú opravami predchádzajúcich pokusov, alebo nové nezávislé pokusy. Pre každý kvíz sa bude určovať, či sa majú sekcie kvízov zamiešať ako aj otázky v nich. Učiteľ zadáva počet pokusov na daný kvíz, koľko minút bude pokus trvať a od kedy do kedy bude otvorený a od kedy bude možnosť pozerať hodnotenie kvízu. Čas prezerania hodnotenia môže nastať ešte pred koncom kvízu, aby sa študenti mohli prípadne opraviť.

Nastavenia kvízu Hodnotenie kvízu sa rozdeľuje na 2 častí. Takýto prístup sme zvolili kvôli tomu, že chceme aby kvízy boli formatívne. Chceme dať možnosť študentom opraviť si pokus na základe spätnej väzby, ktorú dostanú. Čas začiatku opravného obdobia môže učiteľ nastaviť, ak ho ale nenastaví kvíz ho nebude mať. Učiteľ môže nastaviť formu, ktorá sa zobrazí študentom počas opravného obdobia a po uzavretí kvízu. Môže vyberať z možností spätnej väzby ku otázkam, ku možnostiam, počtu dosiahnutých bodov a zobrazenia správnych možností.

Sekcie kvízu Po týchto nastaveniach učiteľ vytvára sekcie kvízu, kde prvá sekcia je automaticky vytvorená a nedá sa vymazať. Do sekcie sa pridávajú otázky buď manuálne alebo náhodne, kde sa dá vybrať kategória, a či daná náhodná otázka môže byť aj z jej podkategórií. Náhodné otázky sme zvolili preto, aby mal každý študent iné otázky ako jeho spolužiaci a musel tak prísť na riešenie sám. Pri veľkom počte otázok môže študent pri každom pokuse dostať mierne odlišné otázky, čo ešte lepšie preverí jeho pripravenosť. Ako pri náhodných tak aj pri manuálne vybratých otázkach sa určuje typ otázky. Pri náhodných je možné vybrať aby aj typ otázky bol náhodný. Učiteľ môže naraz pridať aj viacero náhodných otázok zadaním počtu. Tak isto vie pridať

naraz aj viacero manuálne vybratých otázok zaškrtnutím všetkých otázok, ktoré chce pridať. Manuálne pridané otázky môžu byť pridané naraz aj z viacerých kategórií a rôznych typov.

Keď takto učiteľ vytvorí jednu sekciu, dokáže si rovnakým spôsobom vytvárať ľubovoľný počet sekcií. Každá sekcia okrem prvej sa dá aj vymazať. Otázkam v sekciách vieme meniť poradie.

Uloženie kvízu Po vytvorení celej šablóny a stlačení na tlačidlo, ktoré ju uloží, sa spustí algoritmus, ktorý skontroluje, či je dostatočný počet otázok, ktoré sa môžu vybrať do kvízu tak, aby sa neopakovali. Pokiaľ je dosť otázok, šablóna sa uloží. Inak sa vypíše chybová hláška, ktorá napovie, kde je nedostačujúci počet otázok. Tento problém sa dá riešiť vytvorením ďalších otázok, alebo vymazaním niektorých otázok zo šablóny , ktoré spôsobujú tento problém.

Vypracovanie kvízu

Pri otvorení kvízu na vypracovanie sa ešte raz skontroluje či môže byť vytvorený a ak áno, tak sa spustí algoritmus na generáciu kvízu. Do kvízu sa najprv priradia manuálne pridané otázky, a potom sa z množiny náhodných otázok vyberie vhodná podkategória, ktorá spĺňa podmienky. Keď sa kvíz vygeneruje zobrazí sa študentovi, ktorý ho vypĺňa.

Časovač kvízu Pri kvíze sa zobrazuje časovač, ktorý hovorí koľko je času do konca. Pokiaľ čas vyprší kvíz sa automaticky zavrie. Pokus sa automaticky ukladá každú minútu. Študent vie uložiť aktuálne odpovede aj manuálne bez toho, aby ešte odovzdal kvíz. Po vypršaní času sa kvíz automaticky uloží a zavrie.

Sekcie vygenerovaného kvízu Kvíz je organizovaný do sekcií, kde v každej sekcií je ľubovoľný počet otázok. Študent sa vie medzi sekciami presúvať ľubovoľne, nie je dané presne poradie ako treba kvíz vypĺňať . Otázky sú 3 typov ako bolo spomenuté. Ku každej otázke sa zobrazí prislúchajúce rozhranie, do ktorého sa dá zadávať odpoveď. Študentom je umožnené písať odpovede aj pomocou markdownu a latexu.

Vypracovanie kvízu Študent môže kvíz vypnúť aj počas pokusu, ale čas mu beží ďalej. Vie sa vrátiť aj nazad do daného pokusu a pokračovať v uloženej práci. Po vypršaní časového limitu, aj pokiaľ študent vypol a neuložil kvíz sa automaticky uloží.

Vyhodnotenie kvízu

Študent si vie prezerať vyhodnotenie kvízu, kde sa mu zobrazí spätná väzba podľa toho, ako ho určil autor kvízu.

Správne odpovede

Pri každom type otázky sa zobrazí rozhranie, ktoré prezradí študentovi správnu odpoveď na danú otázku.

Zobrazenie bodov

Zobrazí sa rozhranie, ktoré ukáže celkový počet bodov za daný kvíz a individuálne pre každú otázku, koľko bodov z koľkých študent získal. Otázka je buď zodpovedaná správne, vtedy sa študentovi zobrazí plný počet bodov, alebo nesprávne a vtedy sa zobrazí nulový počet bodov.

Spätná väzba ku otázkam

Pod každou otázkou sa zobrazí globálny komentár učiteľa ku danej otázke. Táto spätná väzba môže byť buď pozitívny v prípade správnej odpovede, alebo negatívny v prípade nesprávnej odpovede. Pri negatívnej spätnej väzbe sa môže nachádzať odkaz na prednášku a podobne, čo má študenta nasmerovať ku správnej odpovedi.

Spätná väzba ku možnostiam

Pod každou možnosťou sa zobrazí krátka spätná väzba , v prípade nesprávnej odpovede.

Rozhranie je rovnaké, ako pre vypĺňanie kvízu, len s rozdielom že možnosť pridávať odpoveď je zamknuté. Učiteľ navyše dokáže v prípade potreby manuálne upravovať bodové hodnotenie daných otázok konkrétnym študentom.

Analýza kvízov

Podobné rozhranie ako pre vyhodnotenie kvízu. Pri každej otázke je krátka analýza, ktorá obsahuje správnu odpoveď, prípadná spätná väzba študentov na danú otázku a najčastejšie nesprávne odpovede. Pri každej otázke je navyše ešte percentuálna úspešnosť danej otázky a počet študentov, ktorý danú otázku dostali do kvízu. V prípade náhodných otázok sa zobrazuje zoznam otázok ktoré boli vybraté algoritmom a rovnaký popis ako je už vyššie spomenuté.

Výsledky kvízov

Rozhranie, v ktorom si učiteľ bude vedieť pozerať výsledky daných kvízov na úrovni študentov. Učiteľ bude vidieť koľko študentov sa zúčastnilo na kvíze a priemerný počet bodov. K dispozícií bude mať tabuľku, kde budú všetci študenti, ktorý sa kvízu zúčastnili a ich hodnotenie za posledný pokus. Učiteľ sa vie dostať aj do kvízov konkrétnych študentov cez toto rozhranie a upravovať hodnotenie manuálne.

4.1.3 Správa študentov

Študenti sa do aplikácie prihlasujú pomocou svojho Github-ového účtu. Tento prístup sa zvolil kvôli tomu že do aplikácie Logic Workbook sa prihlasuje tiež pomocou Githubu, a aby bol tento prístup jednotný. Rovnaká autentifikácia neskôr uľahčí implementáciu modulu priamo do Logic Workbook-u.

Zoznam študentov

Každý učiteľ má prístup ku správe študentov. Vidí zoznam učiteľov a študentov a v prípade potreby vie meniť status študenta na učiteľa a naopak. Používateľov si dokáže filtrovať podľa mena a tak isto aj triediť.

Hodnotenie študentov

Na hodnotenie študentov bude v systéme zabudovaný samostatný modul, ktorého úlohou bude hodnotiť kvízy, a podľa toho im prisudzovať body.

Body sa budú prideľovať systémom 0% alebo 100%, čo znamená že buď mám celú otázku správne zodpovedanú, tak dostanem body, alebo nemám celú otázku správne zodpovedanú a nedostanem body.

Učiteľ bude mať možnosť toto hodnotenie upraviť aj manuálne pre každého študenta zvlášť v rozhraní na to určenom. Ako dokáže hodnotenie prisudzovať systém aj učiteľ, tak to dokáže aj študent.

Systém umožní študentovi počas vypĺňania kvízu posielať spätnú väzbu ku každej otázke jeho kvízu zvlášť v prípade nejasností. Táto spätná väzba sa potom dostane ku učiteľovi a dokáže spracovať dané pripomienky ku otázkam, a zmeniť ich.

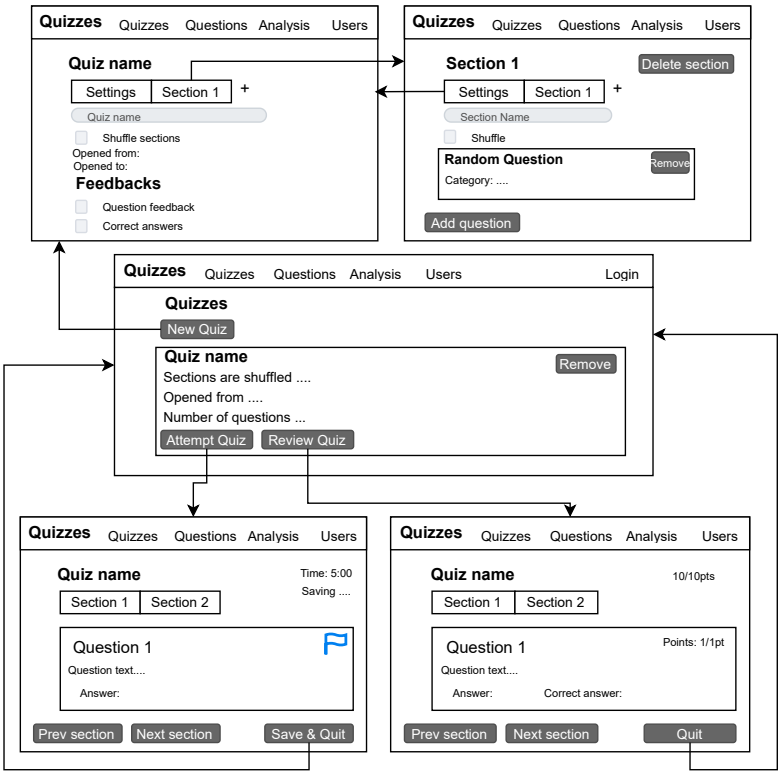
Tak isto študent dostáva spätnú väzbu od učiteľa, ktorý ho pri vytváraní otázky vytvorí a systém ho potom zobrazí pri review študentovi.

4.1.4 Vývojové prostredie

Ako sme spomínali v kapitole 2, tak na vývoj backendu budeme používať Python, konkrétne framework Flask a na vývoj frontendu budeme používať HTML, CSS a využijeme konkrétne štýly z bootstrapu. Kvôli tomu, že chceme vytvoriť modulárnu a dynamickú stránku, ako hlavný jazyk na frontend využijeme React. Na databázu sme zvolili PostgreSQL.

4.2 Používateľské rozhranie

Náčrt používateľského rozhrania rozdelíme na 2 obrázky pre prehľadnosť, keďže máme viacero rozhraní. Na týchto obrázkoch bude popísané a označené , návrh, ako sa dá v



Obr. 4.1: Náčrt základného rozhrania

aplikácií pohybovať a pracovať s ňou.

Náčrt pre rozhranie kvízov

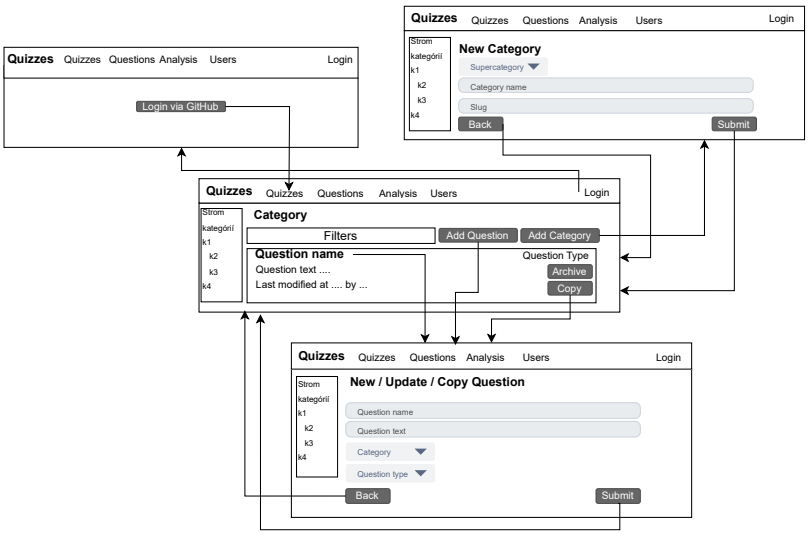
Na obrázku 4.1 je zobrazené, ako sa z hlavného komponentu kvízov vieme dostať do ďalších rozhraní. Do týchto rozhraní sa dostávame cez tlačidlá, ktoré aj popisujú čo v danom rozhraní vieme robiť a ako s nimi interagovať.

Náčrt pre rozhranie otázok

Na obrázku 4.2 je zobrazené, ako sa z komponentu otázok dá pracovať s otázkami, vytvárať ich a do ktorých rozhraní nás dané tlačidlá presunú. Navyše je tu zobrazené rozhranie pre prihlasovanie do aplikácie, ktoré nás presmeruje do rozhrania kvízov, zobrazeného na obrázku 4.1 .

4.3 Používateľské roly

V tejto sekcii definujeme používateľské roly a ich prístup k funkciám nášho systému. Rozlišujeme tri roly: používateľ, učiteľ a študent, pričom roly učiteľ a študent dedia



Obr. 4.2: Náčrt pre rozhranie otázok

vlastnosti zo základnej roly používateľ. Ich use case diagram je zobrazený na obrázku 4.3 .

4.4 Návrh databázy

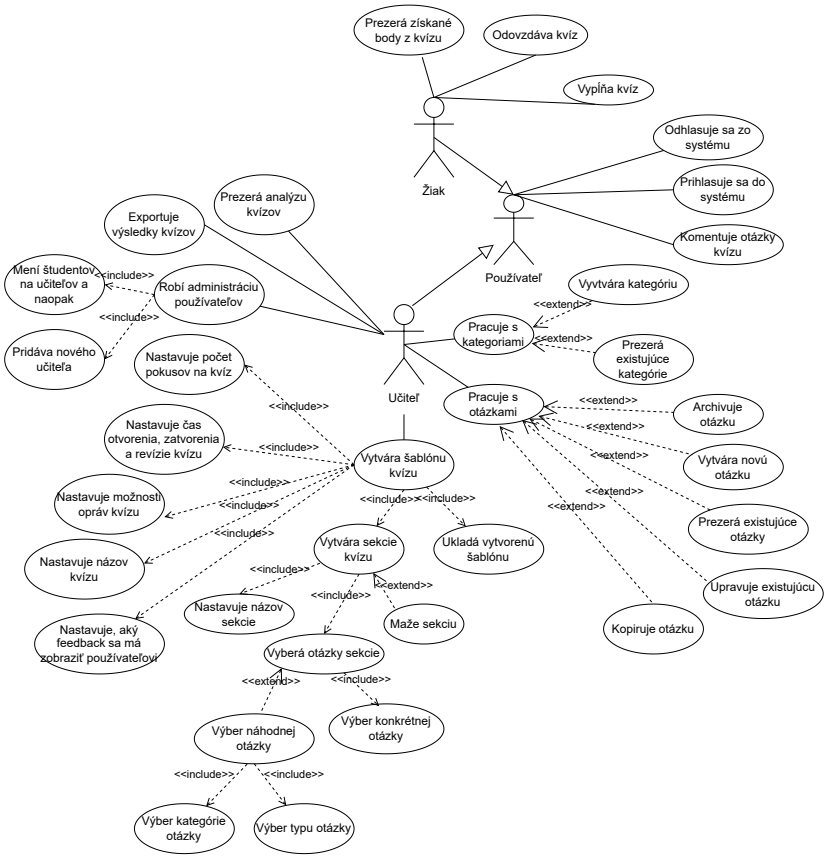
Pri návrhu aplikácie kvízov pre LPI sme prvotne používali databázu mySQL, v ktorej sme ale zistili viacero nedostatkov pre naše potreby, keď mySQL nepodporuje Arrays a databáza sa nedá naindexovať pre rýchlejšie vyhľadávanie v databáze. Preto sme sa rozhodli používať PostgreSQL, ktoré nám toto všetko umožňuje. Náš databázový systém je spravovaný cez admin prostredie pgAdmin 4, vďaka ktorému sa dá databáza ľahko debugovať a meniť podľa potreby.

4.4.1 Dátový model

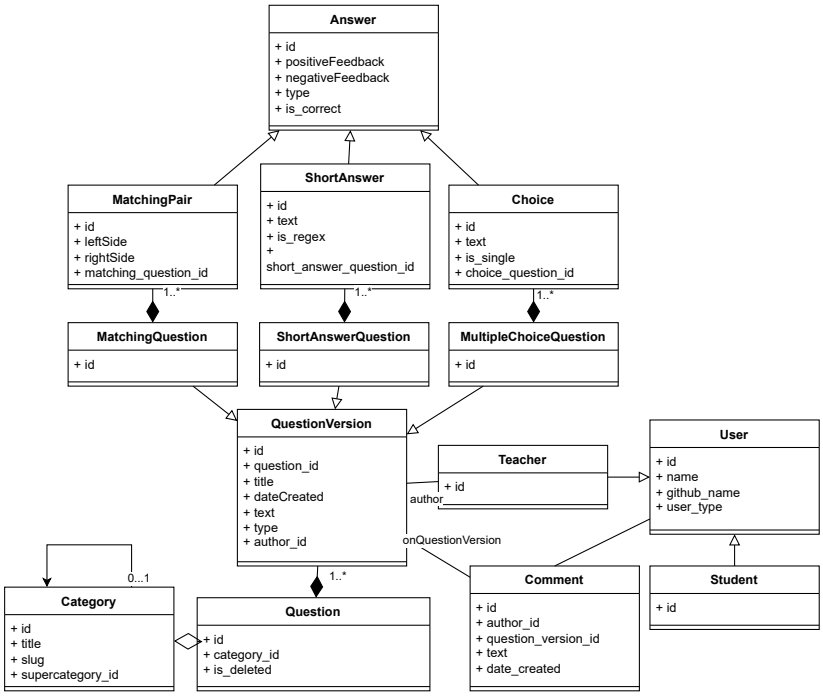
Dátový model prezentujeme ako class diagramy, ktoré sú rozdelené na 2 diagramy. Všetky tabuľky obsahujú svoj unikátny identifikátor. Prvý diagram je triedny diagram pre entitu Questions zobrazený na obrázku 4.4

Dátový model Questions

Question Tabuľka Question je nosná tabuľka v tomto diagrame. Sú v nej uložené všetky otázky, a každá otázka sa viaže na jednu konkrétnu kategóriu. Každú otázku môžeme označiť flagom is_deleted, čím označíme otázku ako archivovanú. Otázka je ďalej viazaná na QuestionVersion tabuľku, čím vieme docieliť, že jedna otázka má viacej verzií. Pri vytvorení novej otázky sa vytvorí sa aj prvotná verzia pre danú otázku.



Obr. 4.3: Use case diagram



Obr. 4.4: Triedny diagram pre Questions

Verzie otázok sa nikdy reálne nemažú, len sa archivujú, lebo by to spôsobovalo nekonzistentnosti v kvízoch ktoré obsahovali dané otázky. Dalo by sa to riešiť aj tak, že stará verzia by sa vždy archivovala a vytvorila sa nová, ale zvolili sme prístup s objektom otázky, lebo si myslíme, že to bude pre používateľa pochopiteľnejšie.

Category Každá kategória má svoj názov, k nemu prislúchajúci slug, čo je verzia názvu vhodná na použitie v URL adresách a môže mať prislúchajúce podkategórie. Slug sa odvodzuje od názvu aplikovaním určitých pravidiel ako sú medzery sa menia na podtržníky, veľké písmená sa menia na malé atď. Kategórie tvoria stromovú štruktúru. Kategória môže byť superkategória čo v skutočnosti znamená, že jej nadkategóriou je kategória supercategory, ktorá slúži ako koreň tohto stromu. Supercategory vzniká automaticky pri prvom spustení aplikácie. Alebo kategória môže byť podkategória, čo znamená, že je viazaná na nejakú superkategóriu, do ktorej podstromu patrí. Týmto spôsobom nám vzniká n-árna stromová štruktúra kategórií. Keď sa kategória vymaže, kaskádovito sa vymažú aj jej podkategórie.

QuestionVersion QuestionVersion je určitá verzia konkrétnej otázky. Táto verzia je vždy vo vzťahu s danou otázkou. Verzia otázky obsahuje nadpis, dátum kedy bola vytvorená, text otázky a typ otázky. Každá otázka sa viaže na autora, ktorý je z tabuľky Teacher. Momentálne môže mať otázka 3 typy, a to krátka odpoveď, spájačka a výber viacerých možností. Vďaka zvolenej štruktúre databázy bude v budúcnosti jednoduché pridať nový typ otázky. Pokiaľ zanikne otázka, ku ktorej verzia patrí, tak zaniknú aj všetky verzie danej otázky. Tento postup nie je štandardný a používateľ nemá rozhranie na skutočné zmazanie z databázy.

MatchingQuestion Táto tabuľka slúži len ako kontajner pre páry spájačky. V praxi to funguje tak, že všetky páry prislúchajúce k jednej spájacej otázke sú viazané na jeden riadok v tabuľke MatchingQuestion a tá je následovne viazaná na verziu otázky. Pokiaľ zanikne tento kontajner pre nejakú konkrétnu otázku, zaniknú aj všetky prislúchajúce páry.

MatchingPair MatchingPair, je jeden pár otázky typu spájačka. Ako atribúty má ľavú stranu a pravú stranu, a viaže sa na MatchingQuestion. Taktiež dedí atribúty z tabuľky Answer, do ktorej sa ukladajú ďalšie údaje o otázke.

ShortAnswerQuestion Kontajner pre krátku odpoveď. Všetky krátke odpovede sú viazane na túto tabuľku, ktorá dedí od tabuľky QuestionVersion.

ShortAnswer Táto tabuľka obsahuje údaje o odpovedí pre krátku odpoveď. obsahuje správnu odpoveď, údaj o tom, či správna odpoveď je v tvare regexu, podľa toho sa potom neskôr vyhodnocujú otázky daného typu. Taktiež dedí atribúty z tabuľky Answer, do ktorej sa ukladajú ďalšie údaje o otázke.

MultipleChoiceQuestion Kontajner pre výber viacerých odpovedí. Všetky možností z tabuľky Choice sú viazané na túto tabuľku, ktorá dedí od QuestionVersion a pokiaľ zanikne údaj v tejto tabuľke, zanikne aj prislúchajúci údaj v tabuľke Choice.

Choice Táto tabuľka obsahuje údaje o konkrétnych možnostiach v otázkach s výberom viacerých odpovedí. Obsahuje údaje ako text možností.

Answer Táto tabuľka obsahuje údaje o tom, či daná možnosť v MultipleChoice je správna, a viaže sa na typ otázky. Ku každej otázke sa môže pridať ešte pozitívnu a negatívnu spätnú väzbu, ktorá sa neskôr v kvíze zobrazuje študentovi po vyplnení kvízu.

Comment Táto tabuľka je spojená s tabuľou QuestionVersion a slúži na to, keď sa študentovi nepozdáva otázka, tak k nej môže pridať komentár a tak isto aj učiteľ. Jeho údaje sa následne uložia v tejto tabuľke aj s dátumom vytvorenia daného komentáru.

Teacher Táto tabuľka dedí z tabuľky Users, a slúži len na to, aby rozlíšila študentov od učiteľov. Konkrétne v tejto tabuľke sa ukladajú učitelia.

Student Táto tabuľka dedí z tabuľky Users, a slúži len na to, aby rozlíšila študentov od učiteľov. Konkrétne v tejto tabuľke sa ukladajú študenti.

User V tejto tabuľke sa ukladajú všetky údaje o používateľoch, či už sú to učitelia alebo študenti. Každý používateľ má svoje meno, a Github-ové meno, ktorý sa ale dokáže vyplniť aj sám pri prihlásení sa používateľa do systému.

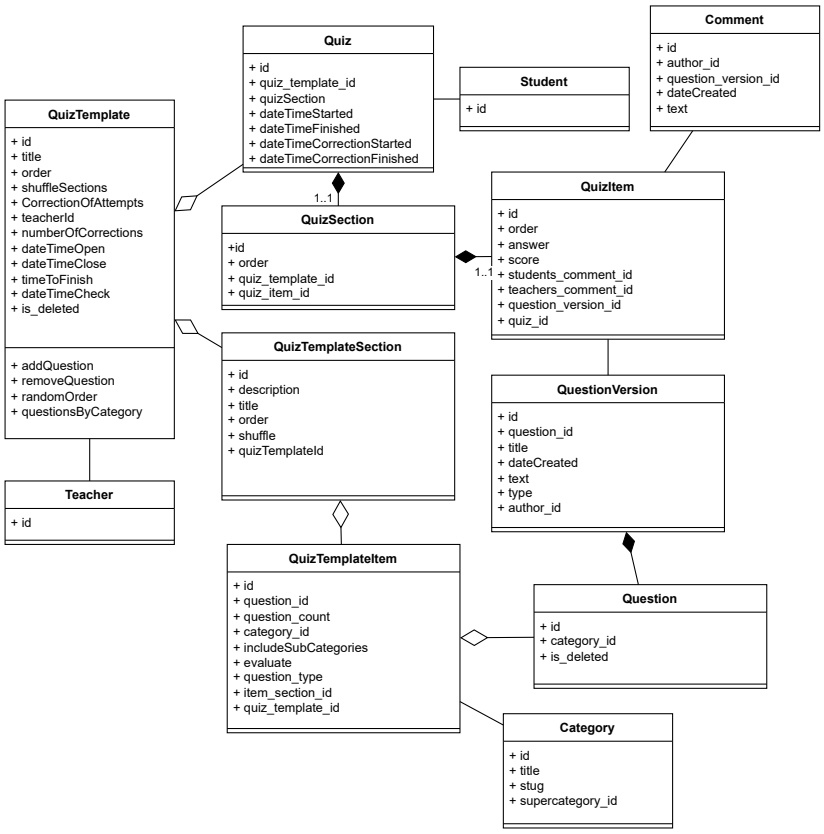
Dátový model Quizzes

Diagram dátového modelu pre kvízy je zobrazený na obrázku 4.5 .

QuizTemplate Táto tabuľka definuje šablónu kvízu, podľa ktorej sa v budúcnosti bude generovať reálny kvíz. V našom prípade je veľmi potrebné mať takúto štruktúru, keďže v našich kvízoch nebudú len otázky manuálne vybraté učiteľom, ale existuje možnosť pridávať náhodné otázky do kvízu na základe nejakého filtra. Každá šablóna musí obsahovať neprázdny názov kvízu, bez názvu kvíz nemôže vzniknúť. Ďalej existuje veľa atribútov, ktoré definujú ako kvíz bude vyzerať, ako aj časové ohraničenia od kedy do kedy sa dá vyplniť a od kedy sa dajú kontrolovať odpovede. Pri atribútoch kvízu je tak isto aj možnosť pre učiteľa, aké údaje v prípade kontroly kvízu žiakom ukazovať. Na výber sú možnosti ako ukázať správne odpovede, ukázať spätnú väzbu ku otázkam, ku možnostiam, alebo ukázať počet bodov za každú otázku. Učiteľ pri vytváraní kvízu tak isto určuje, či sa majú sekcie kvízu zamiešať, ako aj počet pokusov na daný kvíz pre študenta.

QuizTemplateSection Je vždy napojená na tabuľku QuizTemplate, ktorá aj určuje poradie sekcií. Každá sekcia obsahuje ľubovoľný počet QuizItem prvkov, ktoré prezentujú danú otázku. Každá sekcia má názov a možnosť, či zamiešavať otázky v nej alebo nie.

QuizTemplateItem Priamo sa napája na sekciu šablóny a prezentuje jednu otázku, ktorá je buď manuálne vybratá konkrétna otázka, vtedy si pamätá len id danej otázky, a prislúchajúce hodnotenie, ktoré sa vždy zadáva manuálne, alebo je to náhodná otázka



Obr. 4.5: Triedny diagram pre quizzes

a v tom prípade si treba pamätať viacero vecí. Prvou je typ otázky, ako sme spomínali vyššie existujú 3 typy otázok. Keď sa vyberie typ otázky, je potrebné vybrať z akej kategórie má náhodná otázka byť pričom je možnosť nastaviť aj vyberanie otázok aj z podkategórií.

Quiz Každý kvíz obsahuje údaj o šablóne, podľa ktorej sa má vytvoriť. Podľa prislúchajúcej šablóny sa vygeneruje celý kvíz, ktorý obsahuje údaje o otvorení kvízu, keďže každý kvíz má údaj o čase, kedy sa začal a keď sa už kvíz zavrie - vyprší čas na vyplnenie kvízu tak sa spätne vyplní aj tento údaj.

QuizSection Generuje sa na základe prislúchajúcej QuizTemplateSection, ktorá ako šablóna sekcie hovorí, koľko má mať reálny kvíz sekcií a koľko otázok má byť v danej sekcií. Do zobrazeného kvízu zo šablóny preberá všetky potrebne údaje.

QuizItem Je to konkrétna otázka, ktorá prislúcha ku danej otázke zo šablóny. Do tejto tabuľky sa ukladajú všetky údaje od študenta k danej otázke ako sú odpovede študenta, skóre za danú odpoveď, pokiaľ už bol kvíz ukončený, ako aj komentáre od študentov a učiteľov na dané otázky, čím sa zabezpečuje spätná väzba medzi študentmi a učiteľmi.

Kapitola 5

Implementácia

V tejto kapitole sa budeme zaoberať technickými detailami našej aplikácie určenej pre predmet Logika pre informatikov. Predstavíme postup implementácie a technológie. Opíšem dôležité časti aplikácie, ako je užívateľské rozhranie, backend aplikácie, práca s databázou a autentifikácia voči Githubu pomocou OAuth.

5.1 Štruktúra

V tejto kapitole popíšeme štruktúru projektu. Zameriame sa na to, aby sa čitateľ tejto práce vedel zorientovať v tom, ako je systém modulovaný a navrhnutý. Tak isto, keby niekedy v budúcnosti chcel niekto niečo pridávať do projektu, aby dostal rozumnú informáciu o tom, ako je to štruktúrované a ľahko vedel pokračovať vo vývoji.

```
quizzes/
|-- .venv/
|-- backend/
|   |-- functions/
|   |-- migrations/
|   |-- models/
|   |-- routes/
|   |-- static/
|   |-- app.py
|   |-- create_database.py
|   |-- requirements.txt
|   |-- run_migrations.sh
|-- frontend/
|   |-- node_modules/
|   |-- public/
|   |-- src/
```

```
|   |   |-- components/
|   |   |-- Categories/
|   |   |-- Questions/
|   |   |-- Quizzes/
|   |   |-- Users/
|   |   |-- App.js
|   |-- .env
|   |-- .env.template
|-- .env
|-- .env.template
|-- Dockerfile
|-- docker-compose.yml
```

Kód je rozdelený na 2 častí - frontend a backend. Pre úpravu kódu je veľmi dôležité pred spustením správne nastaviť `.env` súbory podľa ich `.env.template` predlohy.

Frontend je naprogramovaný v reacte (3.2) a rozdelený do viacerých častí. V adresári `frontend/src` sú všetky potrebné kódy na to, aby frontend aplikácie fungoval. Je to rozdelené na častí, kde každý názov súboru naznačuje, aký kód sa nachádza v danom súbore. Napríklad v súbore `frontend/src/Questions` sa nachádza kód, ktorý je určený na správu otázok a všetky operácie s nimi. V adresári `.../components` sú uložené komponenty, ktoré sa znovu používajú na viacerých miestach kódu.

Backend je naprogramovaný vo flasku (3.1) delený taktiež do viacerých súborov pre prehľadnosť kódu. V adresári `backend/functions` sú rôzne funkcie, ktoré sú znovu používané na viacerých miestach v kóde, alebo boli moc komplikované na to, aby boli priamo v endpointoch a tým znižovali prehľadnosť kódu. Do adresára `.../migrations` ukladá SQLAlchemy, súbory, ktoré popisujú migrácie databázy po zmene dátového modelu. Tento súbor netreba nijako spravovať ani v ňom nič robiť, kód to robí pri spustení automaticky. V adresári `.../models` sú uložené modely databázy. Flask pracuje s databázou cez ORM, teda každá tabuľka je trieda, a tieto triedy sú uložené v tomto adresári. V adresári `.../routes` sú uložené endpointy, ktoré sú ďalej štruktúrované do `.py` súborov, podľa toho, na čo dané endpointy slúžia. Napríklad pre endpointy, ktoré pracujú s otázkami, existuje súbor `questions.py`, v ktorom sú všetky dané endpointy. Súbory `create_database.py` a `run_migrations.sh` sú automaticky spúšťané súbory pri vytvorení kontajnera, alebo spustení backendu. V app.py je telo Flask projektu, ktoré sa spúšťa a bez ktorého by aplikácia nefungovala.

V koreňovom adresári projektu sa nachádzajú súbory `DockerFile` a `docker-compose.yml`, pomocou ktorých sa dá pre celý projekt vytvoriť kontajner, ktorý beží vo virtuálnom prostredí - sandboxe. Chceme, aby backend aj frontend fungovali na jednom porte. Preto bolo potrebné vytvoriť funkciu na backende, ktorá bude servovať frontend.

5.2 Databáza

Na perzistentné ukladanie dát sme využili relačnú databázu. Relačná databáza je najbežnejšia voľba pri výbere databázy, konkrétne Postgres (3.4).

ORM

Vytváranie a migrácie tabuliek sme realizovali pomocou knižnice SQLAlchemy(2.1). Dátový model, ktorý sme navrhli v podkapitole 4.4.1 sme implementovali ako podtriedy `db.Model` z SqlAlchemy. Napríklad triedu Question implementuje kód:

```
class Question(db.Model):
    __tablename__ = 'questions'
    id = db.Column(db.Integer, primary_key=True)
    category_id = db.Column(db.Integer, db.ForeignKey('categories.id'))
    is_deleted = db.Column(db.Boolean, default=False)
    category = db.relationship("Category", backref='category')
    question_feedback = db.Column(db.Text, default='')
    question_positive_feedback = db.Column(db.Text, default='')

    question_version = db.relationship('QuestionVersion',
        back_populates='questions', cascade='all,delete-orphan')
```

V tomto kóde `__tablename__` definuje názov danej tabuľky. V tomto ORM vieme vytvárať buď stĺpce databázy, alebo vzťahy medzi viacerými tabuľkami, čo sú na pozadí SQL JOIN dotazy, ktoré vďaka tejto štruktúre netreba písať a sú generované na pozadí. Pokiaľ chceme vytvoriť vzťah medzi dvomi tabuľkami taký, že pokiaľ zaniknú dáta v jednej tabuľke, aby zanikli aj v druhej - teda budú vymazané kaskádovito, použijeme atribút pre vzťahy `cascade='all, delete-orphan'`. V prípade, že chceme, aby nejaká tabuľka zdedila všetky atribúty a vzťahy od inej tabuľky, stačí nám ako pri klasických triedach dať túto triedu od ktorej chceme aby dedila do zátvorky za názov tabuľky. Toto sme využili pri všetkých tabuľkách, kde niektoré dedili od db.Model, a niektoré od nami vytvorenej tabuľky. V našom prípade je tam db.Model, čo je univerzálna trieda vytvorená SQLAlchemy, ktorá popisuje určité správanie a funkcionalitu tabuliek.

Migrácie databázy

SQLAlchemy nám poskytuje možnosť využívať SQL migrácie. Tieto migrácie fungujú tak, že pri každej zmene v databáze, v našom prípade v ORM modeli potrebujeme aktualizovať databázu, čo sa robí pomocou migrácií. Postupným spustením príkazov `‘db migrate -m "text"‘` a `‘db update‘` sa databáza aktualizuje. Funguje to tak, že v adresári backendu sa vytvorí adresár `migrations/` , kde sa ukladá Python kód, ktorý vykonáva zmeny na databáze. V prípade že sa nejaká migrácia pokazí, je potrebné postupovať podľa dokumentácie flask-migrate [3]. Všetky verzie databázy sa

dajú upravovať aj ručne v `migrations/`, ale môže to viesť ku chybám a neodporúča sa to, pokiaľ to nie je nevyhnutné. V existujúcom projekte je potrebné mať adresár `migrations/`, bez ktorého sa migrácie nevedia aplikovať a program bude hlásiť chyby. Pokiaľ nemáme tento súbor, môžeme ho vytvoriť pomocou príkazu `flask db init`, ktorý inicializuje migrácie a vytvorí ich podľa modelu.

Pri inicializácií migrácií viac krát môžu nastávať chyby, kedy Flask nevie nájsť danú migráciu a kvôli tomu žiadne migrácie nezbehnú. Flask si ukladá záznam o migrácií do databázy do tabuľky `alembic_version`. Táto tabuľka obsahuje jeden riadok, v ktorom je názov aktuálnej migrácie. Tento riadok sa neodporúča vymazať, aj keď by opravil problém s chybovými hláškami. Odporúčaný spôsob opravenia tejto chyby zahŕňa viacero krokov. Prvým krokom je vymazať `migrations/` pomocou príkazu `rm -rf /migrations`. Následne vymazať záznam z tabuľky `alembic_version` pomocou `DELETE FROM alembic_version`. Keď toto máme, tak je potrebné znova inicializovať migrácie pomocou `flask db init`, následne `flask db migrate -m "migracia"` a nakoniec `flask db upgrade`. Tento spôsob je bezpečný, pretože nemôže dôjsť ku nekonzistentnosti migrácií a aktuálnej verzie migrácie.

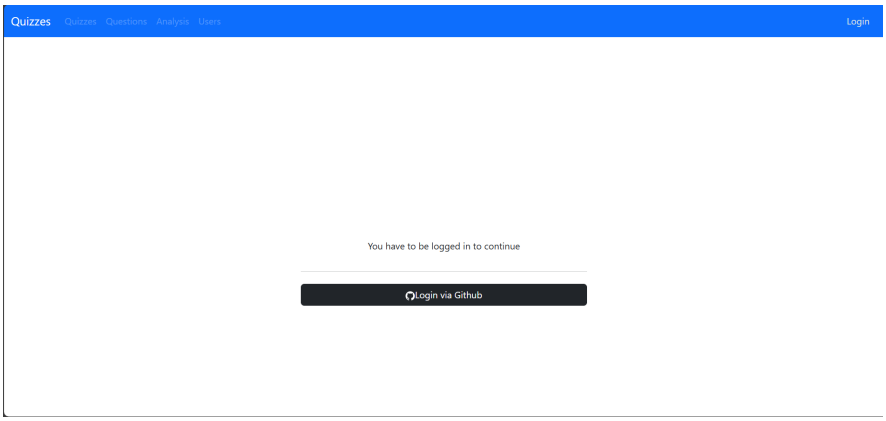
Pokiaľ je možné, pri lokálnom vývoji sa odporúča pomocou spomínaných príkazov migrovať ručne. Obidva spôsoby sú funkčné, ale pri automatickom migrovaní, pokiaľ pracuje viac ľudí na projekte a ukladajú sa aj migrácie na GitHub, môžu nastávať merge konflikty.

5.3 Implementácia používateľského rozhrania

V tejto sekcií sa zameriame na implementáciu rozhrania pre učiteľa a študenta. Poskytneme návod aj s obrázkami z fungujúceho systému, ako sa dá s aplikáciou používať. Aplikácia je nasadená na adrese `https://student.dai.fmph.uniba.sk/quizzes/` . Keď ešte nemám vytvorený účet, aplikácia ma presmeruje priamo do prihlasovacieho formulára.

Prihlásenie do systému

Keď používateľ zapne aplikáciu, začína na stránke s prihlasovaním. Prihlasovanie prebieha pomocou Github účtu, teda nie je potrebné si vytvárať účet v aplikácií. Stačí sa len prihlásiť cez Github účet, a automaticky sa vyplnia potrebné údaje pre používateľa. Pokiaľ ide o inicializáciu aplikácie, prvý používateľ, ktorý sa prihlási do systému dostáva rolu učiteľa, tým sa z neho stáva majiteľ systému a jeho rola sa nedá degradovať na študenta. Autentifikácia s Githubom prebieha tak, že pokiaľ máme prihlásený Github účet, tak tento sa mi automaticky pripojí do systému. Pokiaľ by som chcel prihlásiť iný Github účet, je potrebné ísť na Github a manuálne sa odhlásiť. Keď náš



Obr. 5.1: Prihlásenie do systému

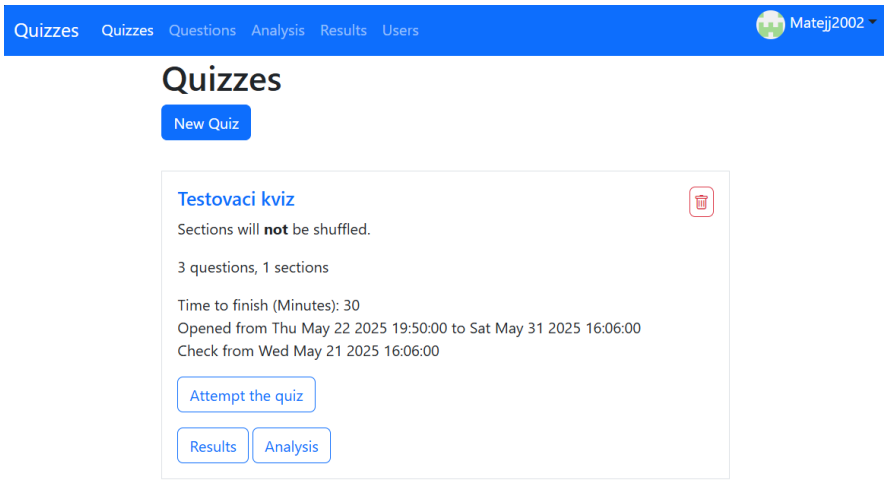
systém nenájde žiadny prihlásený Github účet, presmeruje nás na Github autentifikáciu a umožní nám prihlásiť sa a po prihlásení nás presmeruje späť do našej aplikácie. 5.1

Banka kvízov

Po úspešnom prihlásení, je používateľ automaticky presmerovaný do komponentu kvízov. Kde keď sme prihlásený, vieme vypĺňať prezerať zoznam kvízov, ktoré sú dostupné. Keď je kvíz aktívny je pri ňom aktívne tlačidlo Attempt quiz, ktorým sa kvíz spustí. Pokiaľ v priebehu kvízu kvíz vypneme ale nedáme uložiť finálne odpovede, kvíz je možné spustiť znova s tým že, čas nám beží aj keď sme mimo kvízu. V takomto prípade sa nám zobrazuje tlačidlo Continue current attempt. Po stlačení naň môžeme pokračovať v danom pokuse kvízu až kým nám nevyprší čas. Pokiaľ už kvíz neotvoríme kým vyprší čas, kvíz sa zavrie automaticky. Po celý čas máme možnosť robiť si review starších pokusov daných kvízov. Tie sú uložené v Older attempts a môžeme sa vrátiť ku hociktorému pokusu. Keď sa kvíz zavrie - skončí jeho platnosť, tak sa objaví tlačidlo Review quiz after close, a už není možné otvoriť kvíz na vyplnenie znova. Toto všetko dokáže robiť ako študent tak aj učiteľ, ale učiteľ má ešte práva navyše, keď dokáže archivovať dané kvízy a vytvárať nové šablóny kvízov v hornej časti stránky po stlačení na tlačidlo New Quiz. Učiteľovi sa navyše pri každom kvíze zobrazujú 2 tlačidlá Analysis a Results, pomocou ktorých sa vie dostať priamo do týchto častí aplikácie. 5.2

Vytvorenie novej šablóny kvízu

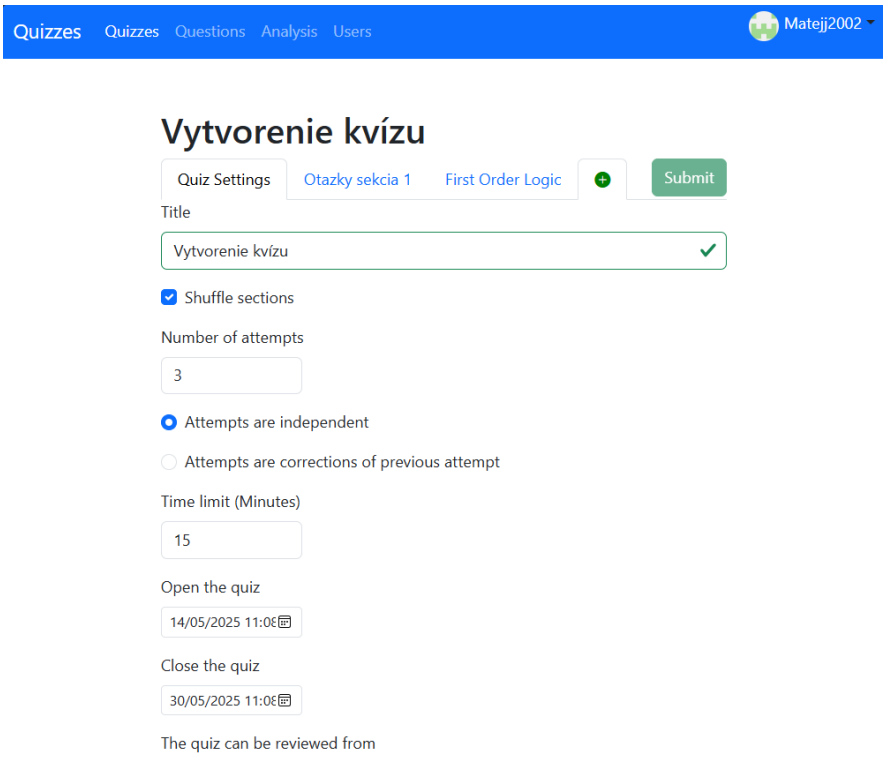
Po stlačení na tlačidlo New Quiz, ku ktorému majú prístup len učitelia, vieme vytvárať novú šablónu kvízu. Vytváranie šablóny je rozdelené do sekcií. Prvá sekcia sú vždy nastavenia kvízu. Tu vieme nastaviť názov kvízu, checkbox na určenie, či sa majú sekcie zamiešavať alebo ostávať v pôvodnom poradí, počet pokusov. Ďalej vieme vybrať



Obr. 5.2: Rozhranie kvízy

či každý ďalší pokus, ako oprava kvízu je oprava predchádzajúceho pokusu, teda kvíz sa negeneruje nanovo, ale ostávajú v ňom rovnaké otázky ako pri prvej generácií. Alebo druhá možnosť, že sa kvíz pri každom otvorení generuje nanovo. Ďalej učiteľ nastavuje čas v minútach na ukončenie kvízu, od kedy do kedy sa kvíz dá vypĺňať a od kedy sa môžu kontrolovať odpovede v kvíze. Pri kontrolovaní odpovedí učiteľ ešte vyberá, aké všetky informácie sa majú študentovi zobraziť, keď si robí review kvízu. Rozlišujeme 2 možností, kontrola kvízu keď ešte kvíz prebieha a kontrola kvízu po ukončení časového rozsahu kvízu. Pre obidve možnosti si vie učiteľ samostatne vybrať, či chce zobrazovať body, spätnú väzbu ku otázkam, ku každej možnosti alebo správne odpovede. Je možné zobrazovať aj viac druhom spätnej väzby naraz, teda môže si vybrať n-rôznych druhov ktoré chce zobraziť. Aj po spustení kvízu vie tieto údaje upravovať podľa potreby hocikedy. 5.3

V prípade zmien v sekciách s otázkami sa neupravuje aktuálna šablóna, ale vytvára sa kópia danej šablóny. Každá šablóna musí mať aspoň jednu sekciu s otázkami, čo je aj základne nastavené a nedá sa vymazať. V každej sekcií vieme pomocou tlačidla nastaviť zamiešanie otázok vrámci danej sekcie. Vieme pridávať buď manuálne vybraté otázky, alebo náhodne otázky, ktoré sa neskôr dajú do kvízu. Pri náhodných otázkach je potrebné určiť typ, akého má byť daná otázka a kategóriu z ktorej vyberať náhodnú otázku. Máme možnosť nastaviť aj vyberanie náhodných otázok z určenej kategórie a všetkých jej podkategórií pomocou checkboxu include subcategories. 5.4 Keď máme vytvorené sekcie, a vybraté otázky do kvízu, môžeme uložiť danú šablónu, ak je to možné - teda je všetko správne vyplnené a tlačidlo na uloženie nie je v stave disabled. Vtedy treba prejsť sekcie a skontrolovať, čo nie je vyplnené. Keď už je všetko správne, tak po uložení šablóny zbehne algoritmus, ktorý skontroluje, či je daný kvíz možné vygenerovať, teda či je dostatočný počet náhodných otázok, ktoré môžu byť zvolené do kvízu a neprekrývajú sa s manuálne vybratými otázkami, aby sa otázky v jednom

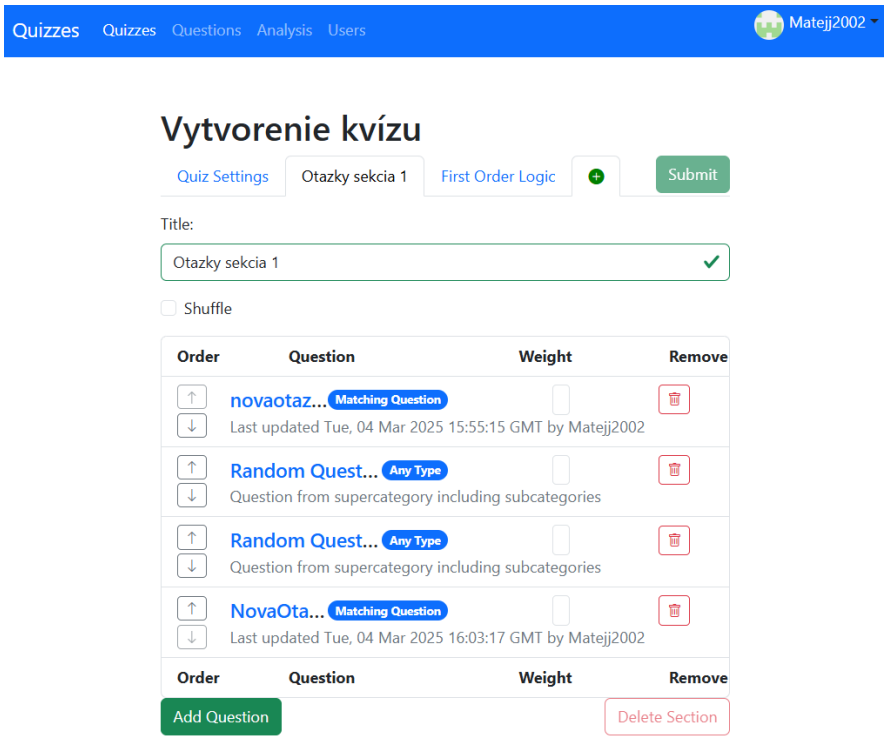


Obr. 5.3: Nastavenia šablóny kvízu

kvíze neopakovali. Pokiaľ nastane chyba, algoritmus vypíše chybu so znením že nemáme dostatočný počet otázok. Ak nenastane chyba, budeme presmerovaný na stránku so zoznamom kvízov, kde už bude aj novo vytvorená šablóna kvízu.

Banka otázok

Pokiaľ nemáme dostatočný počet otázok, alebo chceme vytvoriť nové otázky, presunieme sa pomocou menu do časti Questions. Na tejto stránke vidíme naľavo strom všetkých kategórií, kde každá podkategória je odsunutá o pár prázdnych miest doprava, čo vytvára túto stromovú štruktúru. Pomocou tejto stromovej navigácie kategórií si vieme zobrazovať otázky len určenej kategórie a jej podkategórií kliknutím na niektorú z kategórií. Vtedy sa nám zobrazí v strome celá cesta ku danej kategórií a v strednej časti stránky, kde sa zobrazujú otázky sa načítajú len otázky z časti stromu. Otázky sú stránkované, defaultne je nastavený limit 10 otázok na jednu stránku. V hornej časti otázok si vieme otázky filtrovať na základe toho, či je daná otázka aktívna - teda nie je archivovaná, ako vieme zobrazovať aj archivované otázky alebo v prípade potreby všetky dokopy. Ďalej vieme filtrovať podľa autora otázky a typu otázky. Otázky vieme tak isto sortovať podľa najnovších/najstarších otázok, alebo abecedne. Každá otázka obsahuje názov, na ktorý keď sa stlačí tak sa spustí editor otázok, kde vieme upravovať danú otázku. Ďalej obsahuje typ otázky, text otázky, ktorý ak je dlhý tak sa zobrazí len koľko sa zmestí a metadáta, ktoré obsahujú kedy bola otázka naposledy upravená



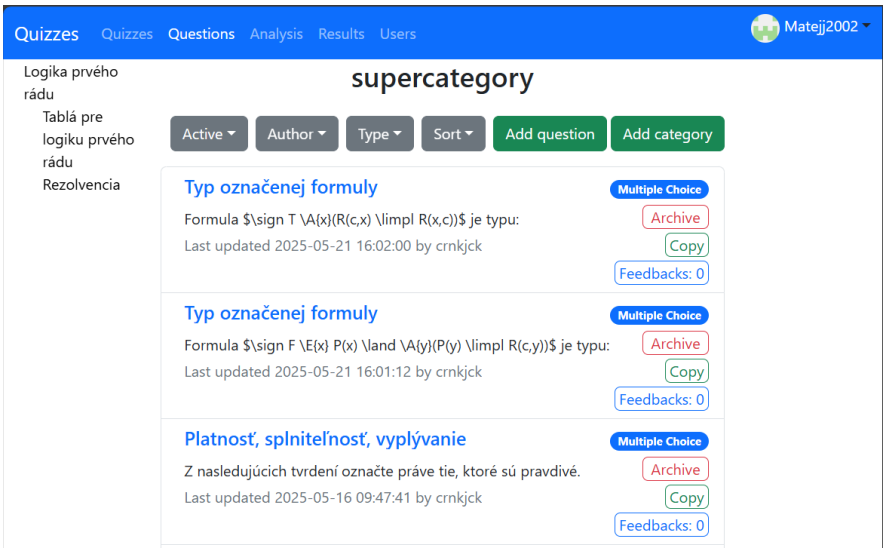
Obr. 5.4: Pridávanie otázok do šablóny

a kto ju upravil. Každú otázku vieme archivovať a pokiaľ je archivovaná tak sa pri nej zobrazuje tlačidlo restore, ktorým ju vieme znova spraviť aktívnou. Otázku vieme aj kopírovať a zobrazovať si spätnú väzbu od študentov pre aktuálnu verziu. 5.5

Tvorba a editácia otázok

Pre vytvorenie novej otázky stlačíme na tlačidlo New Question a zobrazí sa nám rozhranie pre vytváranie otázok. Tu sa dá nastaviť kategóriu do ktorej bude otázka patriť. Táto kategória je prednastavená podľa kategórie, z ktorej sme začali vytvárať otázku. Ďalej sa dá nastaviť typ otázky, podľa ktorého sa nám zobrazí aj spodná časť rozhrania, ktoré sa dynamicky mení podľa toho aký typ otázky sme vybrali. Pri zmene typu otázky sa dáta automaticky nekonvertujú z jedného typu na druhy, takže pri zmene typu otázky prichádza ku strate údajov, ktoré súvisia s typom otázky. Typy otázok sú zadefinované v sekcií 4.1.1. Potom nastavujeme názov otázky a text otázky, na ktorý vieme používať markdown a latex, kde v ľavej časti píšeme text a v pravej častí sa nám zobrazuje generovaný text v markdowne a latexe. Potom už nastavíme len spätná väzba pre študenta v prípade správnej a nesprávnej odpovede a môžeme uložiť otázku.

Pokiaľ chceme editovať existujúcu otázku, tak stlačíme na názov danej otázky a celý formulár sa nám vyplní automaticky. Tu máme možnosť sledovať aj predchádzajúce verzie otázky, ktoré už ale nie sú editovateľné. Vrátiť ku staršej verzií otázky sa dá tak, že si ju nájdeme v zozname a stlačíme tlačidlo Revert to this version, čím sa pridá do



Obr. 5.5: Rozhranie pre evidenciu otázok

zoznamu verzií nová kópia danej verzie navrch a bude aktuálna. 5.6

Tvorba novej kategórie

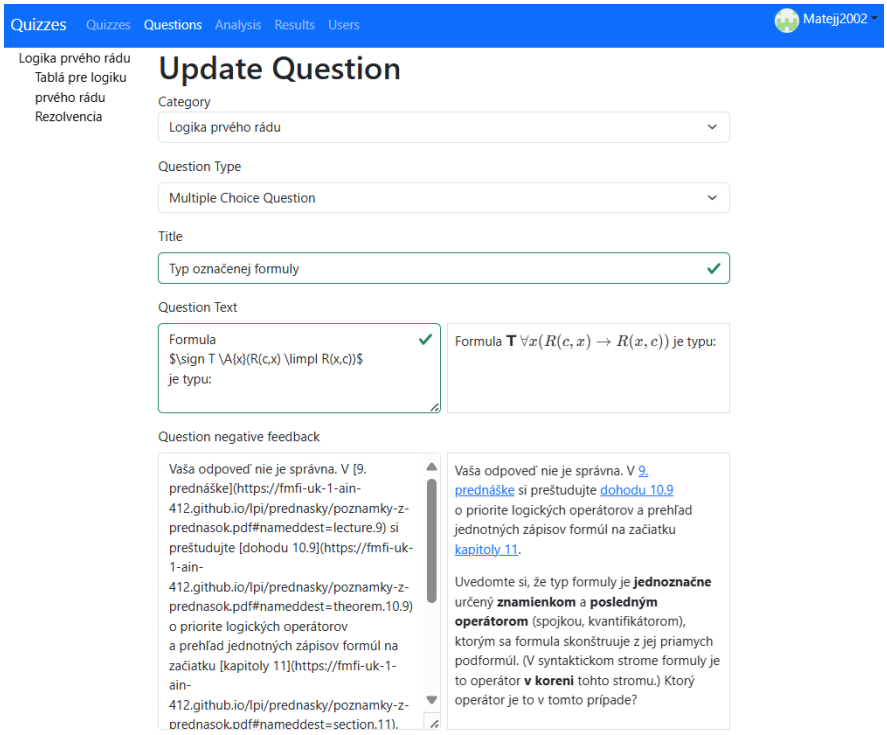
Ak chceme vytvoriť novú kategóriu vieme tak urobiť po stlačení na tlačidlo New Category , kde sa nám zobrazí rozhranie na vytváranie kategórií, kde nám stačí vybrať zo zoznamu kategórií, ku ktorej kategórií bude nová kategória podkategóriou, alebo vieme vytvoriť novú superkategóriu. 5.7

Správa užívateľov

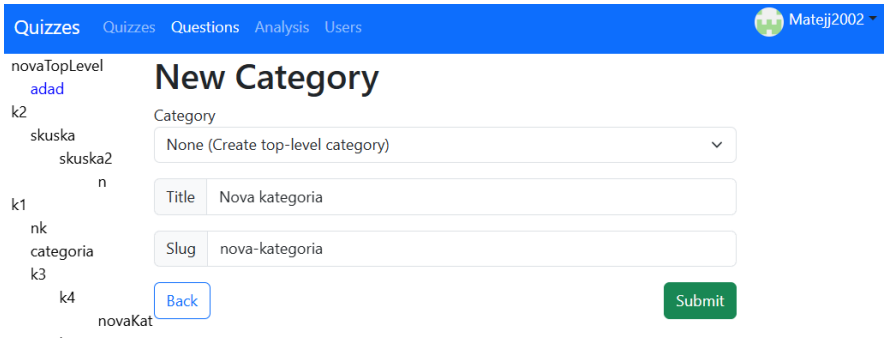
Učiteľ má možnosť spravovať aj používateľov. Zobrazí sa mu tabuľka, v ktorej si vie filtrovať podľa mena a typu a tak isto si vie filtrovať používateľov podľa mena. Každý používateľ má typ, či je študent alebo učiteľ. Každý učiteľ dokáže meniť typ používateľa všetkým, okrem seba a majiteľa systému, aby sa zabránilo tomu, že po nešťastnom kliknutí neostane v systéme ani jeden učiteľ a bola by potreba zasahovať do databázy systému aby sa to opravilo. Učiteľ dokáže pred registrovať ďalšieho učiteľa, ktorý ešte nie je prihlásený v systéme, a ten po prihlásení automaticky bude môcť vykonávať všetko čo môže učiteľ. Existuje možnosť exportu dát, kde po kliknutí na tlačidlo Export Data a vybratí súboru v adresári počítača sa vygeneruje .csv súbor s výsledkami všetkých študentov v aktívnych kvízoch a celkový súčet bodov z kvízov. 5.8

Výsledky konkrétneho študenta v kvízoch

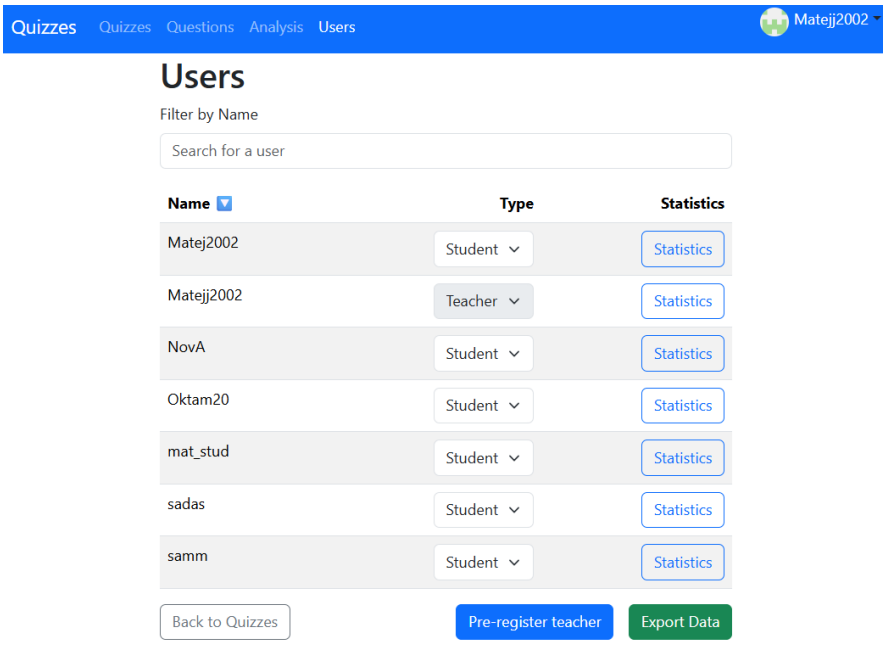
Pri každom používateľovi existuje tlačidlo Review, po ktorom kliknutí sa zobrazí review daného kvízu. 5.9



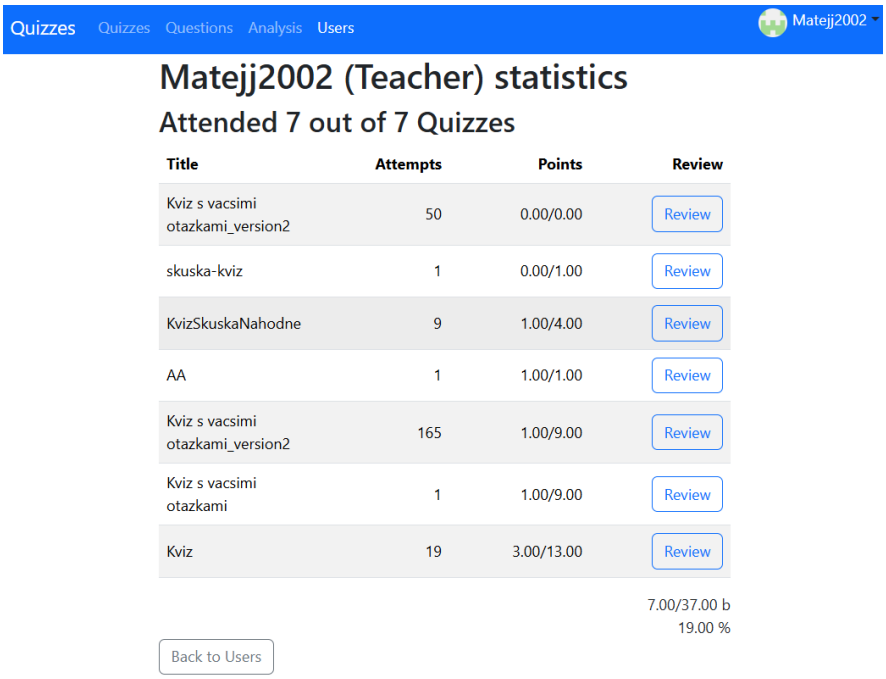
Obr. 5.6: Rozhranie pre vytváranie otázok



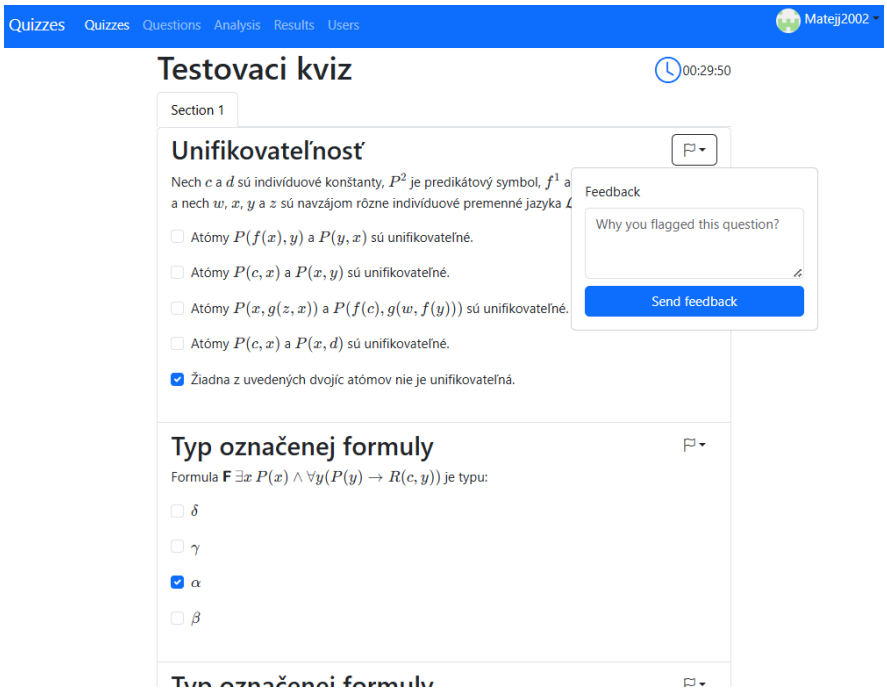
Obr. 5.7: Rozhranie pre vytvorenie kategórie



Obr. 5.8: Správa používateľov



Obr. 5.9: Štatistika študenta



Obr. 5.10: Vygenerovaný kvíz

Vypracovanie kvízu

Ako sme spomínali, študent aj učiteľ vedia vypĺňať kvíz. Toto rozhranie je členené následovne, v hornej častí za zobrazuje názov kvízu a čas ktorý nám ostáva do ukončenia pokusu. Tento čas keď ostáva 5 minút do konca, tak zmení farbu na červenú, nech študenta upozorní na to, že sa blíži koniec času. Každú minútu sa kvíz automaticky uloží, čo sa zobrazí pod časomierou, s textom Saving... . Existuje aj možnosť manuálne uložiť progres kvízu po stlačení na tlačidlo Save v spodnej časti kvízu. Kvíz je rozdelený na sekcie, ako aj šablóna , a v každej sú otázky daných typov so samostatným rozhraním pre každý typ otázky. Ku každej otázke vie študent pridať komentár po kliknutí na vlajočku v pravej časti otázky. Keď sa vyplní celý kvíz, tak na poslednej sekcií je tlačidlo Save and Quit, ktorým sa uloží a ukončí pokus kvízu, ktorý je následne si možné prezrieť s rôznymi radami, ktoré určuje učiteľ. 5.10

Kontrola kvízu

Ako sme spomínali, po vyplnení kvízu sa vie študent presunúť do časti review kvíz, kde si vie skontrolovať odpovede a pozrieť rôzne druhy spätnej väzby ku otázkam. Toto pomáha sa nad otázkou znova zamyslieť a prísť na to po vhodne zvolenej nápovede, radšej ako hneď oznámiť správnu odpoveď študentovi. Študentov to bude motivovať, aby si danú tému ešte raz pozreli a lepšie pochopili pred ďalším pokusom kvízu. V spodnej častí ak je možnosť spustiť kvíz znova, tak sa objaví tlačidlo, ktorým sa dá kvíz znova spustiť. Študenti si budú vedieť pozerať aj svoje staršie pokusy, ktoré sú

Obr. 5.11: Kontrola odpovedí kvízu

označené dátumom, kedy boli odovzdané. V rozhraní pre učiteľa existuje ešte možnosť pozrieť si kvíz v pôvodnom poradí, pokiaľ sa otázky a odpovede v kvíze miešali. 5.11

5.4 Nasadenie aplikácie

V tejto sekcií sa budeme venovať tomu, ako našu aplikáciu nasadiť na server. Aplikácia sa dá nasadiť bez toho, aby bolo potrebné zasahovať do kódu, stačí len správne upraviť `.env` súbory a ďalšie detaily, ktoré popíšeme v tejto kapitole. Kód si naklonujeme pomocou `git clone https://github.com/Matejj2002/quizzes.git`. Po úspešnom naklonovaní repozitára môžeme nastavovať potrebné premenné.

5.4.1 Autentifikácia cez GitHub pomocou OAuth

Keďže sa používatelia autentifikujú pomocou Githubu v aplikácií, je potrebné vytvoriť autentifikáciu pomocou OAuth. Na GitHube v sekcií `https://github.com/settings/developers` je potrebné vytvoriť novú OAuth aplikáciu a nastaviť jej všetky potrebné parametre ako sú Homepage URL čo je URL kde sa naša aplikácia nachádza a Authorization callback URL, kde sa po úspešnom prihlásení do GitHubu používateľ presmeruje, aj s potrebným access tokenom, vďaka ktorému sa vie autentifikovať voči GitHubu a našej aplikácií. Tento Authorization callback URL obsahuje URL, ku ktorej je pridaný endpoint `/login`. Neskôr v aplikácií

budeme používať vygenerovaný `CLIENT_SECRET` a `CLIENT_ID`.

5.4.2 Nastavenie premenných prostredia

V projekte sa nachádzajú dva `.env` súbory. Jeden `.env` súbor sa nachádza v koreni projektu, a slúži pre nastavenie backendu. Druhý `.env` súbor sa nachádza v adresári `frontend/src` a ten slúži pre nastavenie frontendu. Premenné prostredia nastavujeme podľa predlohy `.env.template` a pre frontend je navyše potrebné nastaviť aj súbor `.env.production`, ktorý sa používa pri buildovaní aplikácie. Tento súbor obsahuje presne tie isté premenné ako prislúchajúci `.env` .

5.4.3 Build a compose projektu

Keď už máme nastavenú OAuth a premenné prostredia správne, je potrebné zbuildovať projekt a nasadiť ho na server. Budeme to robiť pomocou `docker-compose` a `dockerFile` súborov, ktoré sú dopredu pripravené. Potrebné je spustiť v koreni projektu príkaz `docker-compose up --build` , ktorý vytvorí kontajner s obrazom pre backend a databázu. Frontend je postavený spolu s backendom ako statický súbor, z ktorého backend vyberá podľa URL adresy pomocou tejto funkcie:

```
@app.route('/', defaults={'path': ''})
@app.route('/<path:path>')
def serve(path):
    if path.startswith(API_URL):
        abort(404)
    if path.startswith("quizzes/"):
        path = path.replace("quizzes/", "", 1)
    if path != "" and os.path.exists(app.static_folder + '/' + path):
        return send_from_directory(app.static_folder, path)
    else:
        return send_from_directory(app.static_folder, 'index.html')
```

Táto funkcia dokáže rozoznávať, ktoré volania idú na statické súbory vygenerované v builde projektu a ktoré volania sú smerované na api.

Podmienkou pre správnu funkčnosť kódu je, že frontend musí byť zbuildovaný, build projektu presunutý do backend súboru do adresáru `backend/static`. Na toto je vytvorený skript v repozitári projektu `backend/setup.py` Keď je toto všetko dodržané, dokážeme spustiť ako aj backend tak aj frontend na jednom porte.

Prvotný compose môže trvať dlhšie, keďže sa musia nainštalovať všetky knižnice do kontajnera. Po nainštalovaní knižníc a nastavení prostredia sa v konzole objavia hlásenia o automaticky spustených migráciách, ktoré buď vytvoria databázu ak ešte nie je vytvorená, alebo len aktualizujú databázu, ak je čo aktualizovať. Ak všetko zbehne bez chybovej hlášky, aplikácia je pripravená na danom porte a môže sa používať.

5.5 Implementácia modulu pre Logic Workbook

Jedným z hlavných cieľov práce, bolo vytvoriť modul, ktorý bude zobrazovať kvízy priamo v aplikácií Logic Workbook. Logic Workbook je aplikácia vytvorená vrámci bakalárskej práce M.Mokom [19]. Vytvorenie modulu si vyžadovalo naštudovanie bakalárskej práce M.Moka a následne aplikovať postup, ktorý je tam spomenutý na pridanie ďalšieho modulu. Na správu stavu sa používa React knižnica redux, ktorá umožňuje ukladať stav aplikácia do store, kde sa uchováva všetok stav.

5.5.1 Postup pridania modulu

Vytvorenie AppComponent.jsx Prvým krokom bolo vytvoriť AppComponent.jsx, ktorý sa bude importovať priamo do Logic Workbooku. Tento komponent, okrem vlastnej definície musí obsahovať aj funkciu configure:

```
export default function configure(backendUrl) {
  return {
    prepare: (initialState, additionalArgs) => {...},
    AppComponent: (props) => (
        <Provider store={props.instance.store}>
          <AppComponent
            instance={props.instance}
            onStateChange={props.onStateChange}
            backendUrl={backendUrl}
          />
        </Provider>
    )}}
```

Keďže náš modul priamo komunikuje s databázou, dôležité je správne nastavenie cesty ku backendu pomocou backendUrl. Do instance sa ukladá stav komponentu, a keď sa jeho stav zmení je potrebné zavolať onStateChange. Toto všetko sa pošle do AppComponent, kde sa definuje správanie komponentu.

Pridanie do hárku Keď máme AppComponent vytvorený, musíme ho pridať do repozitára workbooku. Aplikáciu pridáme následovným spôsobom do `embeddedApps.ts`:

```
{
  name: 'Quiz',
  typeName: 'quizChecker',
  ...AppComponent('http://127.0.0.1:5000/quizzes/api/')
}
```

Keď sme vložili komponent do workbooku, musíme ešte upraviť package.json, kde musíme pridať novú knižnicu, keďže sa očakáva, že naša aplikácia sa nainštaluje ako npm balíček.

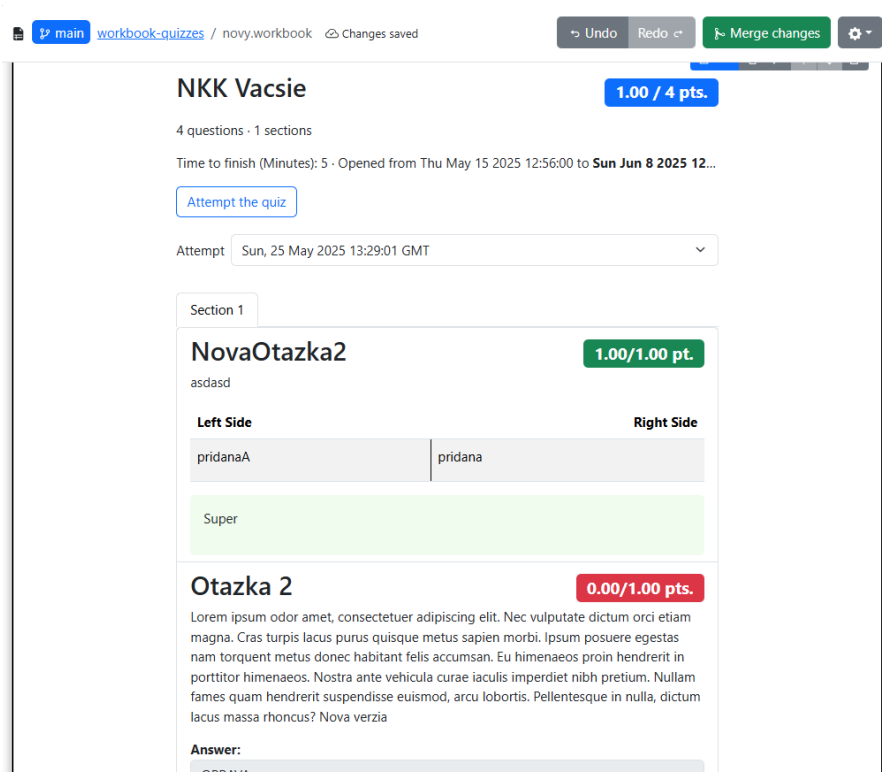
Nastavenie package.json Predtým, ako môžeme vytvoriť npm balíček potrebujeme správne nastaviť package.json, z ktorého npm vytiahne potrebné dáta, viď v kóde.

```
{ "name": "@matejj2002/frontend_quizzes",
  "version": "0.2.91",
  "homepage": "/quizzes",
  "main": "lib/dist/AppComponent.js",
  "types": "lib/types/types.d.ts",
  "files": [
    "lib/**"
  ]}
```

Vytvorenie npm balíčka Na vytvorenie npm balíčka sa musíme nachádzať v adresári, kde sa nachádza frontend našej aplikácie. Potrebujeme mať skript na zbuildovanie react projektu do adresáru `lib/dist`, z kade si ho npm následne vytiahne. Jeden takýto skript sa nachádza v repozitári nášho projektu. Následne sa prihlásime/zaregistrujeme do npm pomocou `npm login` a zdieľame balíček pomocou `npm publish`. Pri každom publishnutí novej verzie je v package.json potrebné zvýšiť premennú "version". Po týchto krokoch si ho už budeme vedieť stiahnuť ako balíček a používať hocikde kde ho nainštalujeme.

5.5.2 Rozhranie pre Logic Workbook

Embedované rozhranie je veľmi podobné rozhraniu v aplikácií. Keďže Logic Workbook funguje na stavoch, nie je možné rútovať modul, preto bolo potrebné vytvoriť samostatnú logiku, ktorá bude prepínať tieto stavy podľa toho, kde sa nachádza používateľ. Keď sa Logic Workbook vypne a znovu zapne, používateľovi sa natiahne posledný stav kde skončil. Toto zabezpečuje react knižnica redux, ktorá si ukladá tieto stavy. V jednom hárku je možné mať aj viacero kvízov naraz, a budú fungovať nezávisle od seba. Rozhranie kombinuje výber kvízu a zobrazenie prehľadu posledného pokusu. Študent si vie prezerať aj staršie pokusy, keď všetky jeho predchádzajúce pokusy sa zobrazujú v zozname a identifikované sú podľa dátumu ukončenia. 5.12



Obr. 5.12: Modul v interaktívnom pracovnom hárku

Kapitola 6

Testovanie

Testovanie aplikácie prebiehalo vrámci predmetu Logika pre informatikov, pre ktorý je náš systém vyvíjaný. Aplikácia bola nasadená na `https://student.dai.fmph.uniba.sk/quizzes` a sprístupnená študentom s možnosťou vyplniť kvíz s možnosťou dvoch pokusov, keď na každý pokus mali 30 minút.

Dotazník sme spracovali na základe štandardu SUS (system usability scale) ktorý pozostáva z 10 otázok a poskytuje rovnováhu medzi kladnými a zápornými otázkami pre čo najpresnejšie výsledky. [18]

Aplikáciu si vyskúšalo 23 študentov. Kvíz vypracovalo 21 z nich a dvaja sa len prihlásili do aplikácie.

Po konci testovania sme žiadali študentov o spätnú väzbu formou dotazníka, na ktorú nám zodpovedalo 12 zo všetkých oslovených študentov. V prvej sekcií dotazníka sme sa pýtali 2 otázky:

1. Ako ste použili kvízový systém?
2. Ak ste niektoré z uvedených funkcií nevyužili, môžete uviesť prečo?

Na prvú otázku nám odpovedali všetci, že vyplnili prvý pokus kvízu a overili si správnosť odpovedí. Možnosť vyplniť kvíz druhý krát využili šiesti študenti a správnosť odpovedí po skončení kvízu si overilo sedem študentov.

Na druhú otázku nám odpovedali traja študenti, že nepotrebovali vyplniť kvíz druhý krát, pretože získali plný počet bodov z prvého pokusu.

V druhej sekcií sme použili štandard SUS, ktorého výsledkom je úroveň použiteľnosti systému v intervale od 0 po 100. Dotazník sa skladá z 10 otázok a na každú otázku je možnosť odpovedať buď úplne nesúhlasím, nesúhlasím, neutrálne, súhlasím alebo úplne súhlasím a sú obodované od 0 po 5, kde úplne súhlasím je za 5 bodov. SUS skóre sa potom vypočíta tak, že skóre pri kladných otázkach odpočítame 1, a od skóre pri negatívnych vypočítame ako 5 - odpoveď. [18] .

Vypočítali sme skóre, ktoré nám vyšlo 76, čo znamená že aplikácia je dobre použiteľná a väčšina používateľov je spokojná, ale stále je tu priestor na zlepšovanie.

Najhoršie skóre z dotazníka mala otázka, ktorá sa pýtala, či je v systéme veľa nekonzistentnosti. Myslíme si, že toto bolo spôsobené tým, že niektoré funkcionality ešte neboli doimplementované počas testovania, čo nám študenti spomenuli aj v tretej otvorenej otázke kde sme sa pýtali študentov, na pripomienky ku systému. Dostali sme pár návrhov na zlepšenie ako sú že v navigácií sa nedalo stlačiť na prepojenie na domovskú stránku aplikácie, ďalej problém s pomenovaním tlačidlá, kde nebolo jasné čo robí- Všetky tieto zmeny sme už implementovali. Celkovo sa odpovede na túto otázku niesli v pozitívnom duchu, keď študenti ocenili prácu a pochválili aplikáciu.

Záver

V rámci tejto práce sme vytvorili kvízovú aplikáciu pre predmet Logika pre informatikov. Vyvíjali sme ju ako webovú aplikáciu, ako aj ako modul do aplikácie Logic Workbook. Aplikácia bola vyvíjaná iteratívno-inkrementálnym prístupom. To znamená, že začali sme programovať, a postupne pridávali nové rozhrania a funkcionality podľa toho ako sme si ich priebežne testovali. Na začiatku vývoja sme sa zamerali na funkcionalitu správy otázok, tvorenia otázok a neskôr sme prešli na implementáciu rozhrania samostatných kvízov. Následne sme pridali autentifikáciu prostredníctvom Githubových účtov a vytvorili ďalšie rozhrania na prezeranie výsledkov z kvízov, analyzovanie výsledkov kvízov. Keď aplikácia bola v stave, že už sa dala testovať reálnymi študentami, nasadili sme ju na školský server a otestovali v rámci predmetu Logika pre informatikov, kde sme zbierali spätnú väzbu od študentov, na základe ktorej sme systém vylepšovali. Okrem samostatnej webovej aplikácie vznikol aj modul pre aplikáciu Logic Workbook, kde sa bude priamo v tejto aplikácií dať spúšťať test a nebude potrebné chodiť na iné stránky. Výsledná verzia aplikácie je plne funkčná a pripravená na využitie vo vzdelávacom procese.

V našej aplikácií je stále priestor na ďalší vývoj a zlepšenia. Napríklad v budúcnosti môže pribudnúť viacero typov úloh, čím by sa ešte zväčšil rozsah a použiteľnosť aplikácie. Možnosť vytvorenia automatického ukladania odpovedí, čo by predišlo strate dát počas vypĺňania kvízov. Momentálne treba ukladať pokus buď manuálne, alebo systém uloží postup každú minútu, čo nemusí stačiť, pokiaľ by systém vypadol medzi uloženiami a dáta s odpoveďami by sa stratili.

Vďaka práci na tejto aplikácií sme nadobudli cenné skúsenosti s modernými programovacími jazykmi a novými prístupmi ku kódu a jeho správe, kde sme využívali Python na postavenie backendu pomocou frameworku Flask a framework React na vybudovanie frontendu. Naučili sme sa aj o tom, ako používať docker na deployment kódu a jednoduchšie nasadzovanie na server, a v neposlednej rade ako pracovať s OAuth v GitHube, pričom podobná autentifikácia funguje aj na ďalších platformách a je v súčastnosti veľmi využívaná.

Literatúra

[1] Dostupná dokumentácia Flask-u , ktorá je online:
https://flask.palletsprojects.com/en/stable/

[2] Dostupná dokumentácia Flask-JWT-Extended , ktorá je online:
https://flask-jwt-extended.readthedocs.io/en/stable/

[3] Dostupná dokumentácia Flask-migrate , ktorá je online:
https://flask-migrate.readthedocs.io/en/latest/

[4] Dostupná dokumentácia SQLAlchemy , ktorá je online:
https://www.sqlalchemy.org/

[5] Dostupná dokumentácia Flask-Admin , ktorá je online:
https://flask-admin.readthedocs.io/en/stable/

[6] Dostupná dokumentácia Django , ktorá je online:
https://docs.djangoproject.com/en/5.2/

[7] História kvízov , ktorá je online:
https://www.britannica.com/topic/quiz/A-history-of-quizzing

[8] Dostupná dokumentácia NextJS , ktorá je online:
https://nextjs.org/docs

[9] Dostupná dokumentácia HTML , ktorá je online:
https://developer.mozilla.org/en-US/docs/Web/HTML

[10] Dostupná dokumentácia CSS , ktorá je online:
https://developer.mozilla.org/en-US/docs/Web/CSS

[11] Dostupná dokumentácia Bootstrap , ktorá je online:
https://getbootstrap.com/docs/5.3/getting-started/introduction/

[12] Dostupná dokumentácia React , ktorá je online:

https://react.dev/learn

[13] Dostupná dokumentácia Docker , ktorá je online:

https://docs.docker.com/

[14] Dostupná dokumentácia Docker , ktorá je online:

https://www.docker.com/resources/what-container/

[15] Dostupná dokumentácia pgAdmin 4 , ktorá je online:

https://www.pgadmin.org/docs/pgadmin4/9.2/index.html

[16] Dostupná dokumentácia PostreSQL , ktorá je online:

https://www.postgresql.org/docs/

[17] Štúdia o dotazníku SUS, ktorá je online:

https://www.researchgate.net/publication/228593520_SUS_A_quick_and_dirty_usability_scale

[18] Dokumentácia Moodle, ktorá je online:

https://docs.moodle.org/500/en/Main_page

[19] Mok, Matej. *Aplikácia na tvorbu a vyhodnocovanie tablových dôkazov.* Bakalárska práca, FMFI UK, 2021. Dostupné online: https://opac.crzp.sk/?fn=detailBiblioForm&sid=72A4E145D65F5DA0BF4D3E0B29C0

[20] Cifra, Milan. *Rezolučný editor pre logiku prvého rádu.* Bakalárska práca, FMFI UK, 2021. Dostupné online: https://opac.crzp.sk/?fn=detailBiblioForm&sid=65CF89236B0A7E124CD77AE040F7

[21] Gombárová, Samantha. *Editor pre formalizačné cvičenia v logike.* Bakalárska práca, FMFI UK, 2022. Dostupné online: https://opac.crzp.sk/?fn=detailBiblioForm&sid=2DF2AC71C78DA39E98FEB4873752

[22] Jurík, Norbert. *Prieskumník štruktúr pre výučbu logiky.* Bakalárska práca, FMFI UK, 2023. Dostupné online: https://opac.crzp.sk/?fn=detailBiblioForm&sid=69B07E5EF4F2B22F0B84B4C7FD1B

[23] Nyitraiová, Alexandra. *Rozšírenia logického hárku pre podporu riešenia dôkazových úloh.* Bakalárska práca, FMFI UK, 2023. Dostupné online: https://opac.crzp.sk/?fn=detailBiblioForm&sid=65CF89236B0A7E124CD67DE040F7

[24] Mgr. Timotej Kubiš *METODIKA TVORBY TESTOVÝCH ÚLOH A TESTOV*. Dostupné online: `https://www.etest.sk/data/files/1917_metodika-tvorby-testovych-uloh-a-testov-kubis-2015.pdf`